Keeping the Referent in the DARK: Anchor-Preserving Token Compression for Privacy-Aware Inference on IoT Logs

Yukun Wen
Department of Computer Science
Khalifa University
Abu Dhabi, United Arab Emirates
100069961@ku.ac.ae

Shijie Zeng
Department of Computer Science
Khalifa University
Abu Dhabi, United Arab Emirates
100065732@ku.ac.ae

***Abstract*—IoT device logs are long, identifier-dense, and routinely copied into language-model prompts. A question about an account names exactly the value that must not reach the model, so the queried entity is also the entity under protection. Redaction removes the value and, with it, the handle the reader needs; compression keeps the handle but drops the record that carries the answer. We show that the two goals need not be traded against each other. In these logs the record that answers the question and the record that stores the protected value are the same block, so DARK keeps that block and aliases the values inside it.**

The rule is training-free: it labels each unit on sensitivity and relevance, applies one of four operations—delete, alias, refill, keep—and writes the same alias map into the question. Relevance is read from the question alone, so the compressor never consults the reference answer.

On 2,728 cases at a 50% budget DARK reaches F1 0.4937, level with the strongest query-aware compressor (0.4831; difference not significant) while leaving 0% of protected values recoverable against that method's 98.1%. It exceeds the only leak-free published baseline by 0.3226 and the strongest leak-free arm of any kind by 0.1456 [+0.1249, +0.1668]; those two arms differ only in which region survives, one line or the block that contains it. On a 112-case set, keeping 48.6% of the tokens is statistically indistinguishable from aliasing the full context without compression (0.6366 vs. 0.6475). Removing the alias step costs 0.3273 F1 where the anchor is itself a protected value and 0.0013 where the question names no entity, and removing the question-side rewrite drives that same subset to exactly 0.0000: the rule acts where designed and is inert elsewhere. The ordering holds at every budget from 10% to 75%, across a second reader and two paraphrases.

***Index Terms*—prompt compression, privacy-preserving inference, large language models, IoT logs, token budget**

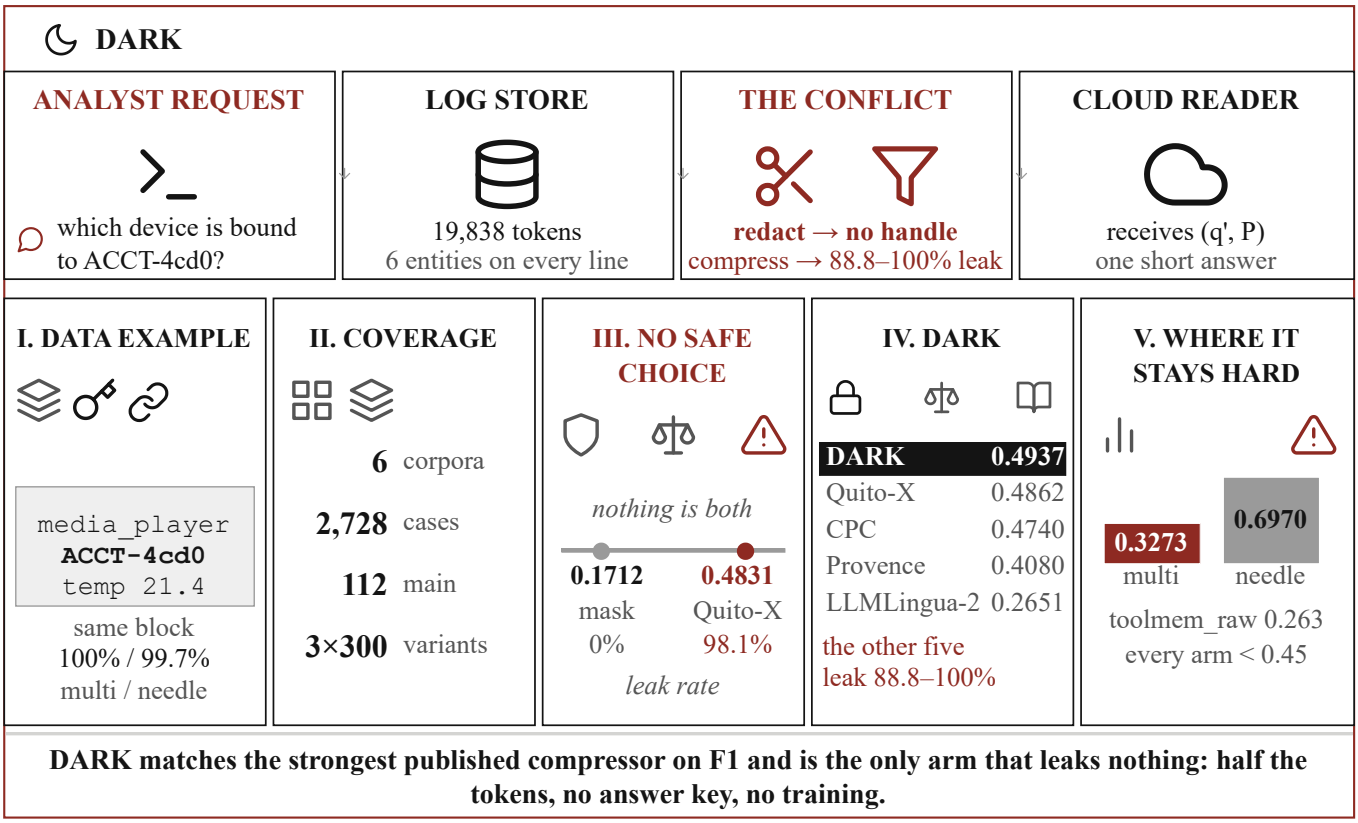


Fig. 1. Why DARK matters. The question names exactly the value that must not be sent, so redaction removes the handle the reader needs while compression forwards the value it must hide; both destroy the task, for opposite reasons. DARK keeps the record that holds the queried entity and aliases the values inside it, which makes it the only arm that is accurate, leak-free and half length at once. The annotation *same block* marks the layout that makes this possible: in 100% of multi-entity and 99.7% of needle cases the record answering the question and the record storing the protected value are the same block. All numbers are from the 2,728-case extension at $r = 0.50$.

I. Introduction

Operational logs from consumer IoT deployments record what each device did, when, and for which account. They are long (an excerpt in our benchmark reaches 19,838 tokens), repetitive, and full of identifiers: account handles, device serials, contact fields. They are also a natural substrate for question answering. An operator asks which device is associated with a given account and expects a short answer.

The property that makes such a question answerable is also what makes it unsafe to send verbatim. The question names an entity, and that entity is itself a protected value. Redacting it before inference is the standard remedy, and it destroys the task: once the account handle is gone, the model cannot tell which of two hundred similar records the question refers to. Compressing the context is the other standard remedy, and it destroys the task for a different reason. Under a redundancy-ordered rule the record carrying the answer survives compression in 47.5% of cases on our benchmark, and in only 37.2% of needle cases, where the answer value repeats across distractor records and the score gives it no priority. Both remedies treat protecting the value and keeping the reference as competing goals to be balanced.

For this class of query, they need not compete. Figure 2 sets out three axes—whether the protected value reaches the model, whether the handle the reader needs survives, and how much of the context crosses. Compression and redaction both halve the context, but compression forwards the value while redaction deletes the handle with it. A fourth route, which we develop here, stops the value and keeps the handle at the same half length. The observation that makes this possible is a layout property. Records in these excerpts are grouped into blocks that hold an entity's identifying line together with the attributes recorded for it, so the queried entity and the answer to the question about it sit in the *same block* in every case, by construction. A pipeline that preserves that block, and replaces the protected values inside it with stable aliases, keeps a referential handle while never exposing the value. Finding the block requires no answer key: the question already names the entity, and the block is whatever contains that mention.

a) What this implies for the design.: A single global decision cannot express it. Aliasing every protected value keeps the handle but spends budget on sensitive records that no question component will use; deleting every protected value removes the handle along with the value. The two axes have to be decided per unit, and the sensitive-and-relevant cell needs an operation that neither redaction nor deletion provides. We therefore route each unit on the pair (sensitivity, relevance) and give each cell its own operation. Sensitive and irrelevant units are *deleted*. Irrelevant insensitive units *refill* whatever budget remains. Relevant insensitive units are *kept* verbatim. Sensitive-and-relevant units are *aliased* rather than deleted. The four operations give the method its name, DARK: **D**elete, **A**lias, **R**efill, **K**eep.

b) Evidence preview.: The design is judged against pipelines that keep the same privacy guarantee, because that is the only comparison in which the privacy operation is held fixed. On a 112-case benchmark spanning six public datasets, DARK uses 48.6% of the original tokens, leaves no protected value recoverable from the prompt, and scores F1 0.6366. Aliasing the whole context without compressing it scores 0.6475 on the same cases, a difference that is not significant, so on this benchmark halving the prompt costs no measurable task quality. The strongest leak-free competitor reaches 0.4308, and in the multi-entity subset it still exposes protected values. On a $2{,}728$-case extension, DARK is level with the best query-aware compressor (0.4937 vs. 0.4831) at 0% leakage against 98.1%: the compressor reaches the same F1 only by forwarding the value the question names, which is the one thing this setting forbids. Removing the alias-instead-of-delete rule costs 0.1795 F1 points overall and 0.3273 on multi-entity cases. On the topical-question variant, where no entity can be located, it costs $+0.0013$ and is not significant. The rule acts where it is designed to act and is inert elsewhere.

c) Contributions.:

- **A failure mode that the compression literature does not report.** When the queried entity is also the protected object, redundancy ordering carries almost no signal about which record the question is about. We quantify it on $2{,}728$ cases (47.5% retention overall, 37.2% on needle cases, both at a budget cut that would give an uninformative rule 50%) and show that protecting the anchor *line* alone repairs only half of the gap (57.6%).
- **A training-free routing rule that closes the gap.** Sensitive-and-relevant units are aliased rather than deleted, and relevance comes from the question by locating the record block that contains the queried entity. The rule keeps a protected value addressable without ever stating it, and needs no reference answer, no training, and no model access.
- **An evaluation that separates privacy from utility at equal budget.** Across two benchmark sizes, three public-benchmark variants, five budgets, two models, and two paraphrases, DARK is compared with six published compressors re-run at full scale plus a masking baseline and a 2026 rule-based compressor, with paired bootstrap intervals on every difference we report. The comparison is deliberately two-sided: at equal budget DARK is level with the best query-aware compressor on F1 and 98.1 points below it on leakage, and $+0.1456$ $[+0.1249, +0.1668]$ above the strongest leak-free arm, which differs from it only in which region of the record survives.

Figure 1 states the setting, the conflict it creates, and the position our rule occupies in one view. Section II positions the work; Section III fixes notation and the threat model; Section IV presents the routing rule; Sections V–VII describe the benchmark and the results; Section VIII isolates each design decision; Section IX states the scope the evidence supports.

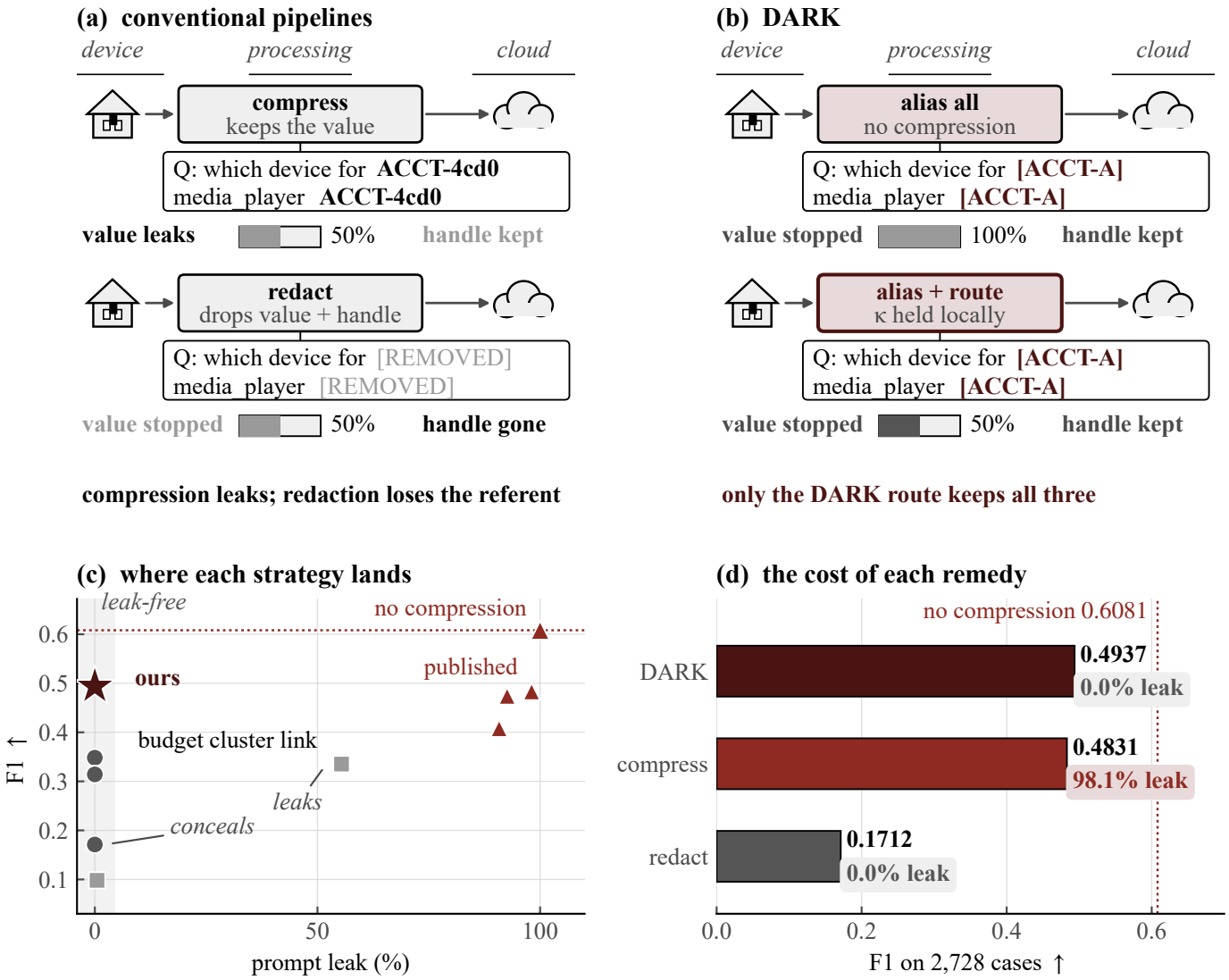


Fig. 2. Three axes, four routes. Under each route the pale box is the prompt as the model receives it—the rewritten question and the record line that carries the answer—and the bar beside the badges is the share of the context that crosses; compressed routes run to the same $r = 0.50$ budget. (a) The conventional remedies. Compressing keeps the handle and halves the context, but forwards the protected value, which the strongest such method leaves recoverable in 98.1% of cases. Redacting removes the value and, with it, the handle the reader needs, so the masking wrapper that does both loses the task. (b) The two routes on our side. Aliasing every protected value stops the value and keeps the handle, but does not compress. DARK stops the value, keeps the handle, and fits the same budget. Only the fourth route holds all three. (c) F1 against prompt leakage for ten arms on the 2,728-case extension. The published compressors forward the value in 88.8% to 100% of cases, the other leak-free arms land between 0.147 and 0.348, and only the marked point reaches 0.494 without leaking. (d) The three policies on the same cases: DARK matches the strongest published compressor on F1, but that arm forwards a protected value in 98.1% of cases, while the redaction baseline reaches 0% leakage at 0.1712.

II. Related Work

A. Prompt compression

One family reduces prompt length by scoring and removing tokens or sentences. LLMLingua [1] and LongLLMLingua [2] use a small language model to estimate token informativeness; LLMLingua-2 [3] distils the decision into a token classifier for task-agnostic compression; Selective Context [4] drops spans that carry little self-information; Provence [5] prunes sentences jointly with question relevance; CPC [6] scores sentences with a context-aware encoder; Quito-X [7] casts compression as an information-bottleneck problem and approximates mutual information with cross-attention; DAC [8] adds an attention-aware term to an entropy criterion; the Perception Compressor [9] retrieves relevant demonstrations with guiding questions before compressing at the token level. These methods differ mainly in how they estimate relevance and redundancy. They share one assumption: that the prompt is ordinary text whose only cost is length. None of them alters a value the model is allowed to read, and none is evaluated on questions whose answer *is* a protected value. The omission is consequential. The strongest of these methods leaves 98.1% of protected values recoverable in our benchmark, and it preserves the answer for a query that names a sensitive entity far less

reliably than it preserves an ordinary answer span.

A second design point compresses into continuous space instead of deleting tokens. Gist tokens [10] and in-context autoencoders [11] train the reader to consume a short learned prefix in place of the prompt, AutoCompressors [12] summarise the context into soft prompt vectors, xRAG [13] distils a retrieved passage into a single token, and COCOM [14] inserts sentinel tokens that stand for a span. Retrieval-side variants compress before the prompt is assembled at all [15], [16], and a parallel line reduces the KV cache rather than the input [17], [18]. Both lines raise the same question for us: what happens to a value that the question needs to name. Recent work measures how much information these compressors actually preserve [19] and finds the loss is not uniform across token types, which is consistent with what we report for identifier-dense text.

A 2026 line takes the opposite route to scoring: instead of a forward pass, it uses linguistic rules—filler removal, contraction, stop-word pruning, lemmatisation, synonym shortening—and is competitive under light to moderate compression at a fraction of the cost [20], [21]. Rules of this kind assume prose, and Section VIII shows what they do to records.

B. Privacy-preserving inference

A second family rewrites protected spans before the prompt leaves the user side. DYNTEXT [22] substitutes sensitive tokens with semantically similar non-sensitive neighbours drawn from a dynamic adjacency list, so that a fixed substitution table cannot be attacked; INTACT [23] replaces identifiers with generalised candidates selected for resistance to inference attacks; PrivacyMind [24] trains a context-level privacy-preserving reader; PAPILLON [25] keeps private spans on a local model and sends only the rest to the cloud; Portcullis [26] and RemoteRAG [27] interpose a sanitising gateway between the operator and a third-party inference service. Masking wrappers around published compressors belong here too. These methods decide per span what to hide and treat utility as whatever survives the rewrite. On our benchmark that framing has a measurable cost: a masking pipeline that achieves zero leakage scores 0.1712 on the $2{,}728$-case extension, because the span it removes is the referent the question needs.

A third body of work asks what a provider can recover from a prompt it receives and retains. ProPILE [28] probes personally identifiable information directly; Staab et al. [29] show that a reader infers attributes that were never stated; ReCaLL [30] and the lower-ranked-token attack of Zhou et al. [31] mount membership inference; Debenedetti et al. [32] catalogue side channels in deployed systems; Split-and-Denoise [33] perturbs client-side representations under local differential privacy; and the survey of Das et al. [34] places these results in one frame. Our threat model is the narrowest of them: we ask only whether the protected value is recoverable from the prompt text, and we claim no resistance to attribute inference or to a provider that manipulates the reader.

A separate line supplies formal guarantees rather than empirical measurements. Anonymisation of published microdata is surveyed from k-anonymity through l-diversity and t-closeness [35]; differential privacy is surveyed across the deployment scenarios in which it is actually applied [36]; and the prompt-specific end of this line covers privacy engineering for prompts in general [37] and utility-aware prompt sanitisation that, like us, trades on which span to replace [38]. Each of these bounds what an adversary can infer from a fixed release. None of them addresses what happens to the answer when the release must still identify one record per query, which is the behaviour Section IX returns to.

C. IoT logs and long-context question answering

The records we compress are device logs, and the IoT literature protects them at other layers: probabilistic encoding for aggregate analytics [39], privacy-preserving data mining with an incentive mechanism [40], asynchronous federated learning over distributed devices [41], privacy-enhanced role-based access control [42], fine-grained cloud-edge data sharing [43], and location-aware offloading that keeps raw data at the edge [44]. Log analytics on this data is mature as well, including online detection under drift [45] and cross-system detection over latent representations [46], but both consume the log in place. Our unit of protection is neither the aggregate, the transport, nor the access rule; it is the prompt that an operator asks about a record, and that unit is what these mechanisms leave uncovered.

LongBench [47] supplies the passages from which we derive the public-benchmark variants, and the standard SQuAD v2 script supplies the scoring function. The evaluation side of this literature has since moved to harder and longer tasks [48], to stress tests that separate genuine long-context ability from retrieval at the right offset [49], [50], and to contexts beyond one hundred thousand tokens [51]. Our contribution is orthogonal to the retrieval and long-context modelling literature: we do not modify the reader, only the text it receives, and we measure the effect at a fixed token budget.

The families above optimise different objectives—length, concealment, and the leakage of everything else a reader can infer—and each reports utility on prompts whose answer is ordinary text. We study the case where both objectives fall on the same span, and show that the resulting conflict is resolvable by changing *which operation* acts on that span rather than by trading one objective against the other. We keep the compression interface of the first family and the per-span rewriting of the second, and modify the stage at which the operation is chosen.

III. Problem Formulation

a) Setting.: A prompt is a pair (q, C): a question q written on the user side and an excerpt C drawn from an IoT log. The excerpt is split into units $U(C) = \{u_1, \ldots, u_m\}$, one per record or key–value group. Each case carries a set of protected values $V = \{v_1, \ldots, v_n\}$; a unit is *sensitive* when it contains a value in V. A model M answers q from the compressed context, and the answer is scored against a reference span.

b) What must not reach the model.: The prompt is the object of study. It travels from the device to the model process and may be logged, cached, or inspected. A policy is *leak-free* on a case when no value in V is recoverable from the exact text handed to the model, that is, from the rewritten question q' concatenated with the compressed context, not from the context alone. Recoverability is measured with the pattern set of the published protocol (full value plus fragments). We assume no adversarial manipulation of the model and no side channel beyond the prompt text, and we do not model an adversary who holds the alias map; Section IX states what that leaves out.

c) The queried entity.: Each question names an entity $e^\star$, which may be an account, a device, or a record identifier. The *anchor* $A \subseteq U(C)$ is the set of units containing a mention of $e^\star$. A unit plays a referential role when the question cannot be resolved without knowing which entity a value belongs to.

d) Budget.: A budget arm receives a ratio r and must realise a compressed context of at most $B = \lceil r \cdot n_C \rceil$ tokens, where n_C is the token length of the uncompressed excerpt under the tokenizer used to build the prompt. Exceeding B is a hard error rather than a silent truncation, because in this domain the answer region is often at the end of the excerpt and truncation would remove it without notice.

e) Metrics.:

- **Task quality.** EM and F1 from the standard SQuAD v2 scorer; UNKNOWN is not a legal answer.
- **Prompt leak**. Fraction of cases in which any protected value is recoverable from question plus compressed context. Lower is better.
- **Gold-in-context (GIC)**. Fraction of cases in which the reference span survives compression. It is model-free, and therefore an upper bound on what the reader can achieve; it is also blind to whether the surviving span is still interpretable, which is why we report task quality as a separate endpoint.
- **Referential integrity**. A case has it when the answer span survives *and* the surviving prompt still tells the reader which record the answer belongs to. It is the conjunction that matters, not either half: a unit can keep the answer and still destroy the task by collapsing every sensitive value onto one surface string, so that the reader cannot tell which record is being asked about. We score the two halves separately—answer survival as GIC, collapse as the collision indicator—because they fail independently and one of them (Figure 7c) is what separates aliasing from deletion.
- **Overshoot**. Indicator that a budget arm exceeded B on a case.

f) The condition we study.: The setting is narrow and we state it as such: long, identifier-dense operational text; a question that names an entity; and a requirement that the entity's value not reach the model. Questions of this form are common in IoT operations and, as Section VIII shows, are exactly the ones on which redundancy-based compression is harmful. The same routing reduces to ordinary compression when the question is topical and names no entity; we report that variant as out of scope rather than as a success.

Algorithm 1 DARK prompt construction

Require: excerpt C, question q, budget ratio r, protected values V

$U \leftarrow \textsc{Split}(C)$
$A \leftarrow \{u \in U : u \text{ mentions } e^\star(q)\}$
$A \leftarrow \textsc{RecordBlock}(U, A)$ ▷ if markers exist
for all $u \in U$ **do**
  if $u \in A$ **then**
    if u is sensitive **then**
      alias(u) ▷ sensitive and relevant
    else
      keep(u) ▷ relevant only
    end if
  else if u is sensitive **then**
    delete(u) ▷ sensitive and irrelevant
  else
    mark u as refill
  end if
end for
$P \leftarrow \textsc{Fit}(\text{kept} \cup \text{aliased}, \text{refill}, B)$ ▷ hard B; degrade, not fail
$q' \leftarrow \textsc{AliasQuestion}(q)$ with the same map
return (q', P)

IV. Method

A. Overview

The design follows from one observation: the unit that must be hidden and the unit that carries the answer are the same record. Any operation applied to that record therefore has to do two things at once—keep it addressable, and stop it from stating the value—and none of the operations a compressor normally has does both. The rule below adds one that does, and routes every other unit to the cheapest operation that applies.

Given an excerpt C, a question q, a budget ratio r, and the set V of protected values, DARK produces a rewritten question q' and a compressed context P such that (q', P) fits the token budget and contains no value of V. The pipeline has four steps: split C into units; label each unit on sensitivity and on relevance; apply the operation attached to its cell; and fit the surviving units into the budget, applying the alias map to the question at the end. The four operations give the method its name: **D**elete, **A**lias, **R**efill and **K**eep. Algorithm 1 states the procedure. There is no training and no model access at this stage. The only deployment-specific input is V, which an operator already holds for redaction. Figure 3 shows the pipeline end to end and the evidence for its one non-obvious choice.

B. Relevance from the question

The relevance label decides which region must survive and which may be spent on the budget. We use one signal: the

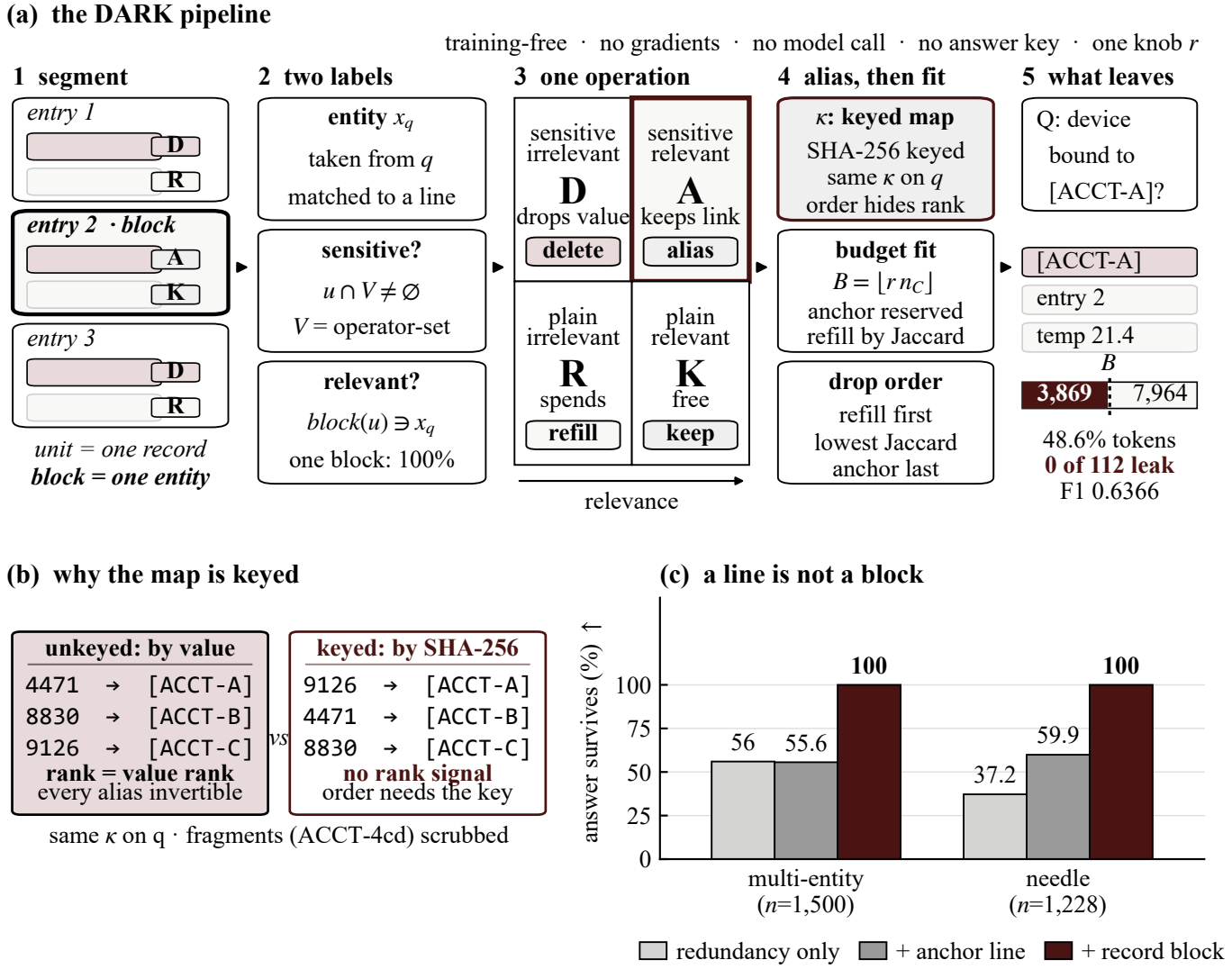


Fig. 3. The DARK pipeline. (a) Five stages, all deterministic and model-free. (1) Units are lines, grouped into per-entity blocks. (2) The question names an entity x_q, matched to a line; sensitivity comes from the protected set V the operator holds, relevance from whether the block contains x_q—no answer key and no model call enters either. (3) The pair selects one operation; only the alias cell keeps the value out and the referent in. (4) A keyed map κ aliases the protected values and is applied to the question as well; the entity block is never dropped, and the rest is fitted to $B = \lfloor r\, n_C \rfloor$ in the stated drop order, refill admitted by decreasing redundancy. (5) A real 112-case run: 3,869 tokens against 7,964, no protected value in any prompt, F1 0.6366. (b) Why the alias map is keyed. Alias letters handed out in value order are invertible from their rank alone; deriving the order from a key removes that signal, and the question is rewritten through the same map so the two halves still agree. (c) Protecting the anchor *line* does not close the gap; protecting the *block* does. Bars give the fraction of the 2,728 cases in which the answer-bearing record survives.

region that mentions the entity the question names. The signal takes two forms in our excerpts.

a) Record blocks.: When the excerpt is segmented into blocks headed by a record marker, the whole block that contains the anchor mention is relevant. This is the common case for log-shaped benchmarks: the benchmark's own annotation places the reference answer inside the same block as the anchor mention in every one of its cases (Table I). A rule defined on blocks therefore keeps the answer without inspecting it.

b) Sentence windows.: LongBench-style passages carry no block markers. There the relevant region is the anchor sentence plus one sentence on each side: five units, about 2.3% of the budget in our measurements. The window is a fallback, not the main mechanism, and Section VIII reports where it suffices and where it does not.

c) No answer key.: A natural implementation of a relevance label is an oracle: mark the unit that contains the reference answer and deprioritise it last when dropping units. That implementation is not deployable, and it changes what the method is. We avoid it, and Section VIII measures how much a pipeline gains from having it, so that the comparison against an oracle-assisted pipeline is explicit rather than implicit.

C. The four operations and the rewritten question

Sensitivity is a property of the unit given V: a unit is sensitive when it contains a protected value. Crossing the two labels

gives four cells, and each cell receives the operation that lets the reader resolve the question while the value stays out of the prompt.

*a) Sensitive and relevant: alias (**A**).:* The unit remains in the context, and each of its protected values is replaced by a stable alias from a per-run mapping, so that repeated mentions of one entity receive one token. This cell is where we depart from prior practice. Redaction, the standard primitive, gives the same concealment but removes the referent. Keeping the unit and renaming its values preserves the association between the entity and its attributes while satisfying the constraint on V. Section VIII reports the cost of replacing this operation by deletion: 0.1795 F1 points overall and 0.3273 on multi-entity cases.

*b) Sensitive and irrelevant: delete (**D**).:* The unit holds a protected value that no question component refers to. It contributes nothing to the answer, so it is removed rather than aliased. This is the only cell in which a protected value disappears from the prompt entirely.

*c) Relevant and insensitive: keep (**K**).:* The unit bears on the question and holds nothing that must be hidden, so it is kept verbatim.

*d) Irrelevant and insensitive: refill (**R**).:* Remaining units hold no protected value and lie outside the relevant region. They compete for whatever budget is left, ordered by the redundancy score of Section IV-D.

Aliasing the context alone produces a prompt whose two halves use different names for the same entity: the context refers to `[ACCOUNT-A]` while the question still names `ACCT-4cd02ed8fb3a`. The reader cannot connect the two, and a string-presence metric reports the entity as preserved while the task is destroyed. We therefore rewrite the question with the same map, so that every mention of the queried entity carries the alias used for it inside the context. The object handed to the model is the pair (q', P), and both halves are written with one map. Section VIII measures the contribution of this step.

The alias assignment also determines whether a reader can invert the map by rank. We fix it per run from a keyed permutation instead of by value order; an unkeyed assignment lets an adversary who knows the value set recover every alias from its rank alone. All reported numbers use the keyed assignment.

D. Budget accounting

The budget is $B = \lceil r \cdot n_C \rceil$, and it is enforced rather than approximated. The surviving set is assembled greedily and then trimmed against the *actual* concatenated text, because the separator between units costs tokens: a sum of per-unit token counts is a lower bound on the length of the joined prompt. When the trimmed set still exceeds B, units are discarded in a fixed priority—refill first, then non-anchor relevant units, then anchor units—so that the pipeline degrades instead of failing. Every budget arm reported in this paper has zero overshoot.

Refill ordering uses a model-free redundancy score: for unit u_i, the maximum Jaccard similarity between its token set and that of any other unit. Candidates are visited in *increasing* redundancy, which keeps the repetitive records.

That direction deserves comment, because it inverts the usual reading of the score. Visiting units in decreasing redundancy and excluding them is the principled interpretation of a redundancy measure, and it is what a standard budget-fitting routine does. Here it is the wrong choice: the answer-bearing record is frequently the most redundant unit, because the answer value repeats across distractors. Section VIII reports both variants and the 0.6567-point difference they produce on the two-hop benchmark. We state the choice as an empirical regularity with a stated mechanism, not as a consequence of the definition.

V. Benchmark Construction and Validation

A. Sources and excerpts

We build the benchmark from six public IoT and smart-home corpora that appear in the retrieval and long-context literature, all redistributed for research use: `pec_home`, `privacylens`, `simuhome`, `smartbench_tdcs`, `toolmem` and `toolmem_raw`. For each source we assemble excerpts in which records for several entities of the same type are interleaved, and we inject synthetic protected values into the records that already exist, rather than into separate carrier documents. Table I reports what each source contributes: the $2{,}728$ cases run from a median of $2{,}151$ tokens (`pec_home`) to $18{,}239$ (`smartbench_tdcs`), with a median of 4.27 record blocks per case.

The excerpts differ in a way that matters for what the rule can be expected to do. `simuhome` has the fewest blocks per case (3.03) and `pec_home` the smallest blocks ($1{,}444$ characters at the median), while `toolmem_raw` carries the largest blocks ($8{,}894$) against a comparable token budget. Section VIII shows that block size, not token count, is what bounds the rule.

B. Protected values

The injected values are canaries, not real personal data: public corpora cannot carry real PII, and a canary makes leakage measurable exactly. Each case carries 12.8 of them on average, drawn in equal thirds from three families—account handles, contact addresses and phone numbers—for $34{,}917$ values in total. Every value has a *marker* (the part that identifies it) and a *fragment length*: account and address markers are twelve hex characters with fragment length six, phone markers are eleven digits with fragment length eight. The two-part design is what lets Section III count whole-value and fragment exposure separately instead of collapsing them into one bit.

The values come from a pool: there are $1{,}781$ distinct values, so the same account handle recurs in about twenty cases. This is deliberate and it is uncomfortable for a design that pseudonymises globally—a single cloud provider seeing many prompts could link occurrences of the same handle—so DARK assigns aliases per prompt rather than maintaining a global map. Section VIII reports the consequence for cross-prompt linkage as a threat-model boundary rather than a guarantee.

TABLE I

WHAT THE EXTENSION BENCHMARK CONTAINS, PER SOURCE. EVERY COLUMN IS RECOMPUTED FROM THE CASE ANNOTATIONS AND THE PER-CASE RECORDS, NOT INFERRED FROM THE CASE COUNTS. *Tokens* IS THE MEDIAN CONTEXT LENGTH UNDER THE READER'S TOKENIZER, TAKEN FROM THE UNCOMPRESSED ARM; *records* IS THE MEAN NUMBER OF ENTRY BLOCKS PER CASE; *block* IS THE MEDIAN CHARACTER LENGTH OF A BLOCK; *values* IS THE MEAN NUMBER OF PROTECTED VALUES INJECTED PER CASE. *Co-located* IS THE SHARE OF CASES WHOSE ANNOTATION PLACES THE REFERENCE ANSWER IN THE BLOCK THAT MENTIONS THE QUERIED ENTITY: IT IS 100% IN EVERY SOURCE, BECAUSE THE CONSTRUCTOR BUILDS THE BENCHMARK THAT WAY. THAT IS A DESIGN CONDITION, NOT A FINDING ABOUT LOGS IN THE WILD—SECTION V STATES WHY IT HOLDS AND SECTION VIII MEASURES WHAT BREAKS WHEN IT DOES NOT.

Source	Cases	Tokens	Records/case	Block (chars)	Values/case	Co-located
`pec_home`	426	2,151	5.01	1,444	15.0	100%
`privacylens`	600	5,024	4.50	4,994	13.5	100%
`simuhome`	312	6,090	3.03	6,138	9.1	100%
`smartbench_tdcs`	490	18,239	4.61	8,138	13.8	100%
`toolmem`	300	7,061	4.00	6,670	12.0	100%
`toolmem_raw`	600	10,808	4.00	8,894	12.0	100%
overall	2,728	7,005	4.27	—	12.8	100%

The consequence of using canaries is that our privacy claim concerns the recoverability of *these* values, not of arbitrary personal information that may co-occur in real logs. We state this here and again where the claim is made.

C. Questions and answers

Each case poses an entity-resolving question. The 2,728 cases are generated from 74 distinct templates; the spread comes from the attribute being asked about, not from paraphrase. Two families of excerpts are included. In *multi-entity* excerpts (1,500 cases) the same question template applies to six accounts and the target account is distinguished only by its identifier. In *needle* excerpts (1,228 cases) the question names a specific record identifier, or an entry index, and asks for a field value inside it; the anchor takes six distinct surface forms across the sources, which we list and verify in the appendix.

Reference answers are derived deterministically from the injected structure, so no answer is authored by hand, and the derivation leaves no ambiguity to score: in all 2,728 cases the answer string occurs inside exactly one record block. This is checked rather than assumed, and it is the reason an exact-match reading of the score is meaningful here.

D. The layout condition

The benchmark is constructed so that the record which answers the question is the record that mentions the queried entity. Table I reports this as *co-located*, and it is 100% in every source. We want to be explicit that this is a *design condition of the construction*, not a discovery about logs in the wild: it is what an operational log looks like when records are grouped by the entity they describe, and it is the condition under which the routing rule is defined. Section VIII measures what happens as the condition weakens—larger blocks, answers outside the fallback window, and the two-hop variant in which the answer sits six sentences from the anchor.

Within that condition the two case families differ structurally, and the difference is the mechanism: on multi-entity cases the anchor and the answer are always in the same block and never on the same line, while on needle cases they usually share the line. That asymmetry is measured directly in Figure 7.

E. Sizes, splits and key hygiene

We report three sets. The *main* set contains 112 cases, 20 from each of five sources and 12 from a sixth, and is used for the main table, the budget curve, the cross-model and the paraphrasing experiments. The *extension* set contains 2,728 cases from the same six sources and is used to check that the main-set conclusions survive an order-of-magnitude increase in size and a different source mix. The six sources contribute 426, 600, 312, 490, 300 and 600 cases respectively, so no single source exceeds 22% of the extension. Both sets share the same question templates.

A note on key hygiene: the two sets reuse the same case identifiers for different content, and joining results across them by identifier silently mixes records. Every analysis reported here (i) resolves results within a single set, and (ii) verifies the join by comparing the uncompressed-context arm against a reference value computed from the same set. The check is reported alongside each table.

F. Public-benchmark variants

To test the rule outside our own construction we derive three variants from LongBench passages, 300 cases each, with the same injected canaries. *Topical* leaves the original questions, which name no entity. *Single-hop* replaces the question with one that names an account, and places the answer in the same record as the account. *Two-hop* places the answer six sentences away from the account mention, so that resolving the question requires following the record structure rather than the local window. Section VIII reports all three, including the one we do not claim.

VI. EXPERIMENTAL SETUP

a) Models and decoding.: The main reader is Qwen2.5-7B-Instruct [52]; the cross-model check uses Mistral-7B-Instruct-v0.3 [53]. Both run in fp16 with greedy decoding, 64 new tokens, maximum context 24,576 tokens and a strict policy that aborts rather than truncates when a prompt exceeds it. The random seed is fixed at `20261015`. Scoring uses the official SQuAD v2 script, a fixed, model-independent procedure.

b) Baselines and fairness.: We re-run six published compressors through their reference implementations: LLMLingua-2 [3], LongLLMLingua [2], Selective Context [4], Provence [5], CPC [6], and Quito-X [7]. We also run a 2026 rule-based compressor that needs no model at compression time [21], the release that accompanies the peer-reviewed study of linguistic rules as compressors [20]; it is a preprint arm and is labelled as such. Where a method exposes both a budget-matched and a native configuration we report both; the budget-matched one is used for the headline comparison, since published defaults vary in length. We additionally run a masking wrapper around LLMLingua-2, and our own non-routing arms: `full`, `alias` (alias all protected values, no compression), `delete` (redact all, no compression), `alias_budget` and `alias_budget_link` (global alias, then redundancy-based selection without and with anchor protection), and the corresponding `budget_cluster` pair on raw text.

All compressors receive the question and the context and nothing else. We verified in the runner that the reference answer is never passed to any baseline, and that no baseline consumes the anchor set. This is the fairness condition for the comparison: the contribution we claim is precisely the ability to obtain the referential benefit without the answer key.

c) Budgets and statistics.: Budget arms are anchored at $r = 0.5$ for the main comparison and swept over $r \in \{0.10, 0.15, 0.25, 0.50, 0.75\}$ for the budget curve. Published compressors are evaluated at their own 50% setting, with the token accounting reported in every table so that a method that realises a different length is visible as such.

All comparisons between two arms on the same cases are paired. We report the mean paired difference and a 95% percentile bootstrap interval with $10{,}000$ resamples over cases. A difference is called significant when the interval excludes zero. We deliberately do not attach a significance claim to any comparison across the two benchmark sizes, which draw different cases.

d) Reproducibility controls.: Two controls are run in every batch. First, the uncompressed arm is recomputed; it involves no selection logic, so it must reproduce the reference value, and any deviation indicates that the run differs in model, precision, or data. Second, every budget arm asserts non-overshoot at the case level. Runs that fail either control are discarded rather than reported.

VII. Results

A. Compression costs little

Halving the prompt costs no measurable task quality. At an equal 50% budget, aliasing the whole context without compressing scores 0.6475 and DARK scores 0.6366 at 48.6% of the tokens. The paired difference is -0.0109 with a 95% interval of $[-0.0824, +0.0612]$, which does not exclude zero. Against not compressing and not sanitizing, the deficit is -0.0823 $[-0.1610, -0.0041]$. That gap is the price of the privacy constraint rather than of compression, since the aliasing arm pays it as well (-0.0714 against `full`).

TABLE II

Main comparison on the 112-case benchmark (Qwen2.5-7B-Instruct, $r = 0.50$, seed 20261015). All arms of ours are constructed without access to the reference answer. *tokens* is the mean compressed-context length; *leak* counts cases in which a protected value is recoverable from question plus context. Arrows give the preferred direction. $BCL =$ `budget_cluster_link`, $ABL =$ `alias_budget_link`, $ALB =$ `alias_budget`, $BC =$ `budget_cluster`. Controlled comparison is the first three rows: compressing to 48.6% of the original length costs 0.0109 against the no-compression aliasing arm, which is not significant. The uncompressed arm is the reference value used to verify that a result set belongs to this benchmark.

Arm	F1 ↑	tokens ↓	leak ↓
`full`	0.7189	7964	112/112
`alias`	0.6475	7824	0/112
DARK	**0.6366**	**3869**	**0/112**
`delete`	0.4916	7845	0/112
BCL	0.4308	3683	47/112
ABL	0.4219	3638	0/112
ALB	0.2232	—	0/112
BC	0.2143	—	8/112

Paired differences against DARK: `alias` -0.0109 $[-0.0824, +0.0612]$ (n.s.); `full` -0.0823 $[-0.1610, -0.0041]$; ABL $+0.2147$ $[+0.1224, +0.3085]$.

The comparison specific to our design is the lower block of the table. Two arms are leak-free and compress to nearly the same length: `alias_budget_link`, which aliases globally and protects the anchor line, and DARK. The difference is $+0.2147$ $[+0.1224, +0.3085]$. The two do not differ in the privacy operation—both alias—but in which region survives: one line, or the block containing it.

B. At scale, privacy is the only separation

Table III reports the $2{,}728$-case extension. DARK scores 0.4937, level with the strongest query-aware compressor (Quito-X, 0.4831; difference $+0.0107$, interval $[-0.0091, +0.0306]$) and above CPC (0.4740) and Provence (0.4080). What separates the arms is leakage. Every published compressor leaves protected values in the prompt, in 88.8% to 100% of cases, because concealing them is not among its objectives. DARK leaves none. Among published baselines the only leak-free one is a masking wrapper at 0.1712, which DARK exceeds by $+0.3226$ $[+0.3041, +0.3414]$. The nearest arm of any kind that also leaks nothing is our own global-alias ablation at 0.3482, which DARK exceeds by $+0.1456$ $[+0.1249, +0.1668]$; those two arms differ only in which region survives, one line or the block that contains it, so that margin is the routing rule and nothing else.

The table also carries a 2026 rule-based compressor, run on CPU at its most aggressive preset [21]. It realises $5{,}817$ context tokens—more than we spend—scores 0.3696, and leaks in 100% of cases. Section VIII isolates why: its rules delete the braces and rename the keys that make a record a record.

The aggregate also hides a split. On needle cases, where the question names a record identifier rather than an account, DARK is statistically indistinguishable from not compressing

TABLE III

COMPARISON ON THE 2,728-CASE EXTENSION (QWEN2.5-7B-INSTRUCT, $r = 0.50$). PUBLISHED COMPRESSORS ARE RE-RUN THROUGH THEIR REFERENCE IMPLEMENTATIONS ON THE SAME CASES; ARMS SUFFIXED _NATIVE USE EACH METHOD'S OWN DEFAULT LENGTH. THE SIX PUBLISHED BASELINES APPEARED BETWEEN 2023 AND 2025; LEXCOMP IS A 2026 PREPRINT, RUN THROUGH ITS RELEASED IMPLEMENTATION AT ITS MOST AGGRESSIVE PRESET, AND IS MARKED AS SUCH. *Leak* IS THE FRACTION OF CASES IN WHICH A PROTECTED VALUE IS RECOVERABLE FROM THE PROMPT. DARK AND THE MASKING BASELINE ARE THE ONLY ARMS AT 0%; DARK EXCEEDS THAT BASELINE BY $+0.3226$ $[+0.3041, +0.3414]$. AGAINST THE STRONGEST COMPRESSOR, QUITO-X, THE DIFFERENCE IS $+0.0107$ $[-0.0091, +0.0306]$, I.E. NOT SIGNIFICANT, AT 0% VERSUS 98.1% LEAKAGE. THE UNCOMPRESSED ARM REPRODUCES THE REFERENCE VALUE 0.6078 FROM A SEPARATELY EVALUATED BATCH TO WITHIN ONE CASE IN 2,728.

Method	Year	Family	F1 ↑	leak ↓
`full`	—	uncompressed	0.6081	100.0%
`ext_full`	—	uncompressed (rerun)	0.6078	100.0%
DARK	—	ours (routes)	**0.4937**	**0.0%**
Quito-X	2025	query-aware	0.4831	98.1%
Quito-X `_native`	2025	query-aware	0.4862	98.1%
CPC	2025	query-aware	0.4740	92.6%
CPC `_native`	2025	query-aware	0.4523	93.4%
LongLLMLingua	2024	task-agnostic	0.4230	100.0%
Provence	2025	query-aware	0.4080	90.8%
Provence `_native`	2025	query-aware	0.4047	88.8%
LexComp `_preprint`	2026	lexical rules, CPU	0.3696	100.0%
`alias_budget_link`	—	ours, no answer key, merged cell	0.3482	0.0%
`budget_cluster_link`	—	raw text, anchor kept	0.3352	55.4%
`quad4_del`	—	ours, delete instead of alias	0.3142	0.0%
LLMLingua-2	2024	task-agnostic	0.2651	100.0%
Selective Context	2023	task-agnostic	0.2588	100.0%
LLMLingua-2 `_native`	2024	task-agnostic	0.2433	100.0%
`mask_llmlingua2`	—	sanitize + compress	0.1712	0.0%
`mask_llmlingua2_native`	—	sanitize + compress	0.1663	0.0%
`alias_budget`	—	global alias, no anchor	0.1469	0.0%
`budget_cluster`	—	redundancy only	0.0981	0.5%

at all ($+0.0160$, interval $[-0.0026, +0.0352]$) while sending half the tokens. The whole deficit sits on multi-entity cases (-0.2212), where six accounts share one question template and the question is the only thing that identifies the target.

The reading we take from this table is narrow. A query-aware compressor reaches the same F1 as our rule at the same budget, but only by forwarding the value the question names. When that value must not be forwarded, the compressor loses the task, as the masking row shows, and the routing rule does not.

Figure 4 qualifies the aggregate: the 0.4937 is not uniform across sources. Deleting the anchor unit instead of aliasing it costs almost nothing on `privacylens` (0.467 against 0.437) and collapses `simuhome` (0.619 against 0.029) and `pec_home` (0.676 against 0.274). A mean over six sources is what makes that look like one effect. DARK sits below the uncompressed arm everywhere but is the arm closest to it on four of six sources; on `smartbench_tdcs` the anchor-protecting arm is closer, and on `toolmem` all four arms fall within 0.04 of each other. On `toolmem_raw` every arm is below 0.45, so the source rather than the operation is the bottleneck. The privacy rows are the part that does not vary: the uncompressed prompt exposes a protected value in 100% of cases on every source, DARK and the delete variant in none, and the anchor-protecting arm leaks in five of six because protecting the block keeps the neighbouring units that carry the value.

TABLE IV

BUDGET CURVE ON THE 112-CASE BENCHMARK. EVERY ROW SATISFIES ITS BUDGET WITH ZERO OVERSHOOT. DARK IS THE STRONGEST BUDGET ARM AT ALL FIVE RATIOS; ITS ADVANTAGE NARROWS AS THE BUDGET GROWS, AND ITS OWN SCORE PEAKS AT $r = 0.50$ BECAUSE AT $r = 0.75$ THE ADDITIONAL REFILL TEXT IS NOISE. THE LAST ROW GIVES THE MEAN REALISED CONTEXT LENGTH IN TOKENS.

Arm	@0.10	@0.15	@0.25	@0.50	@0.75
DARK	**0.4357**	**0.4681**	**0.5116**	**0.6366**	**0.6158**
`quad4_del`	0.3297	0.3788	0.3955	0.5205	0.5086
BCL	0.2304	0.2052	0.2310	0.4308	0.5576
ABL	0.2304	0.2052	0.2488	0.4219	0.5512
realised tokens	656	1033	1913	3869	5830

Uncompressed reference: F1 0.7189 at 7964 tokens.

C. Budgets, readers and phrasing

Across five ratios DARK is the strongest budget arm at every point: $+0.205$ at $r = 0.10$, $+0.263$ at $r = 0.25$, $+0.215$ at $r = 0.50$, and $+0.065$ at $r = 0.75$, all at zero leakage. The margin narrows at the loose end, and the reason is the regime rather than the rule. At $r = 0.75$ the budget stops binding: the arms that protect only the anchor line recover from 24.1% retention at $r = 0.10$ to 77.7% here, so they approach the score the routing rule already reaches, and the gap closes. DARK's own score also falls slightly (0.6366 to 0.6158), because the additional refill text admitted at a loose budget is repetitive

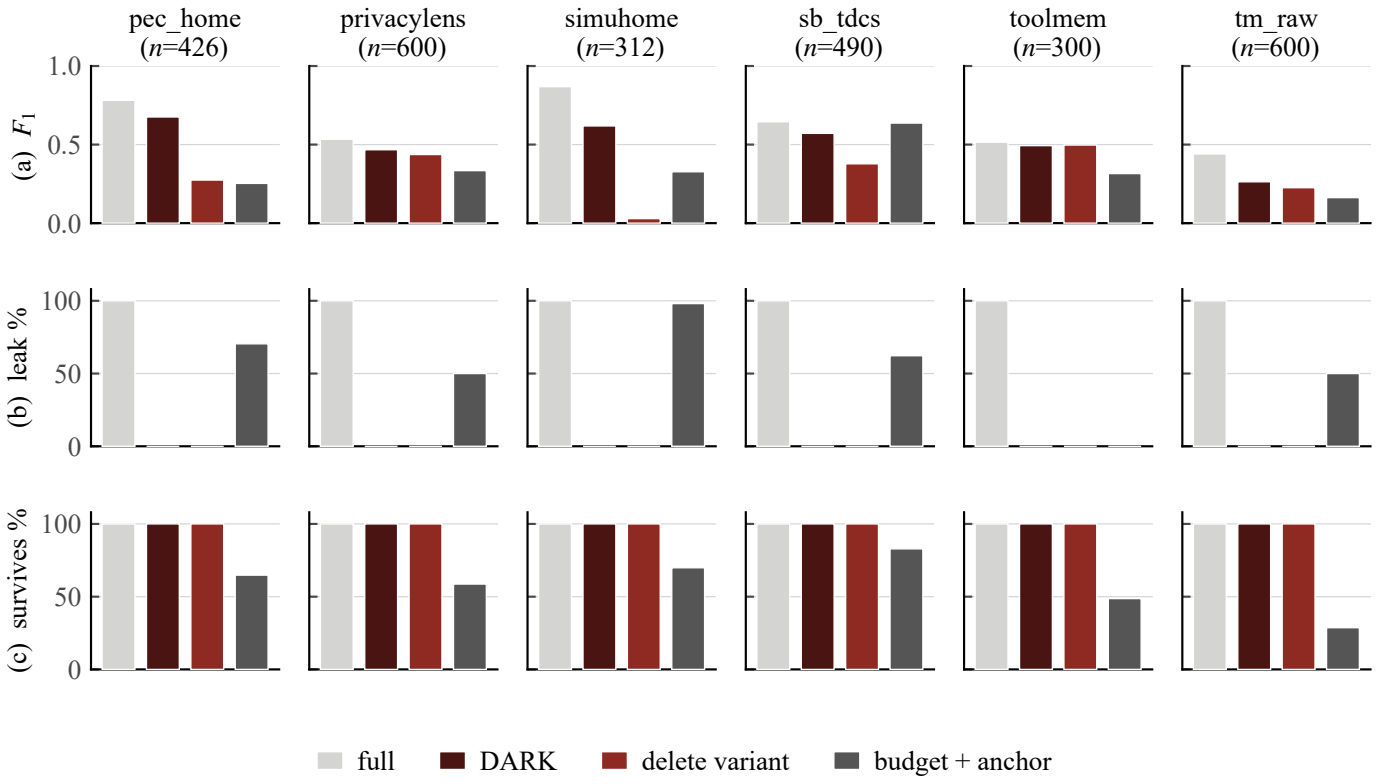


Fig. 4. Per-source behaviour on the 2,728-case extension at $r = 0.50$, one panel per source and one row per metric; n under each column is the number of cases. (a) DARK sits below the uncompressed arm everywhere and is the closest arm to it on four of six sources, but the two effects that matter are local: on `simuhome` the delete variant falls to 0.029 against DARK's 0.619, and on `toolmem_raw` all arms are below 0.45 because the answers themselves are hard. (b) Leakage does not vary by source: the uncompressed arm exposes a protected value in every case, while DARK and the delete variant expose none on any source—a bar of zero height is a zero, not missing data—and the anchor-protecting arm leaks in five of six. (c) The answer-bearing record survives in 100% of cases for the uncompressed arm, DARK and the delete variant, and in 29–83% for the anchor-protecting arm; the whole-vs-line distinction of Figure 3(c) is what separates those three from the fourth.

context that competes with the relevant block for the reader's attention. The setting where the constraint binds is $r = 0.50$, and that is where the mechanism separates most clearly from the alternatives.

On Mistral-7B-Instruct-v0.3 at $r = 0.50$, DARK scores 0.6946 against 0.6366 on Qwen2.5-7B. Every other arm loses ground when the reader changes: `full` −0.0258, `alias` −0.0544, `delete` −0.0357, `alias_budget_link` −0.0471, `budget_cluster_link` −0.0471. The routing rule is the only arm that improves, which is consistent with the mechanism: a second reader gains more from a cleaned, budgeted prompt than from a long one.

Against two mechanical rewritings of the question, DARK scores 0.6637 and 0.6023 against 0.6366 on the original, a maximum change of −0.0343. The uncompressed arm changes by −0.0614 over the same rewritings. The margin over `alias_budget_link` remains significant in all three conditions (+0.2147, +0.2711, +0.1994). The check is relevant because the rule keys on a mention of the queried entity, and a rewriting could in principle disturb that mention; in these variants it does not.

VIII. Analysis

A. Why redundancy ordering fails here

A redundancy ordering keeps the record that carries the answer about half the time. On the 2,728-case extension the redundancy-only rule retains it in 47.5% of cases (56.0% multi-entity, 37.2% needle); protecting the anchor line raises this to 57.6%, and routing on the record block reaches 100%. The question has to be asked in its own terms because aggregate F1 hides it: every arm here produces an answer, and the score does not say whether the record that grounds it is still in the prompt.

The two failure rates say different things. Multi-entity survival sits either side of the budget cut, which is what an uninformative ordering produces: the score does not know which record the question is about, so it keeps the right one about half the time. Needle survival is *below* the cut, and there the ordering is worse than uninformative. The answer in a needle case is a field value that recurs across distractor records, so the record holding it looks like a typical record and inherits no priority from the score. Protecting the anchor line repairs this for needle cases—retention rises from 37.2% to 59.9%—but the line it protects is not the line that carries

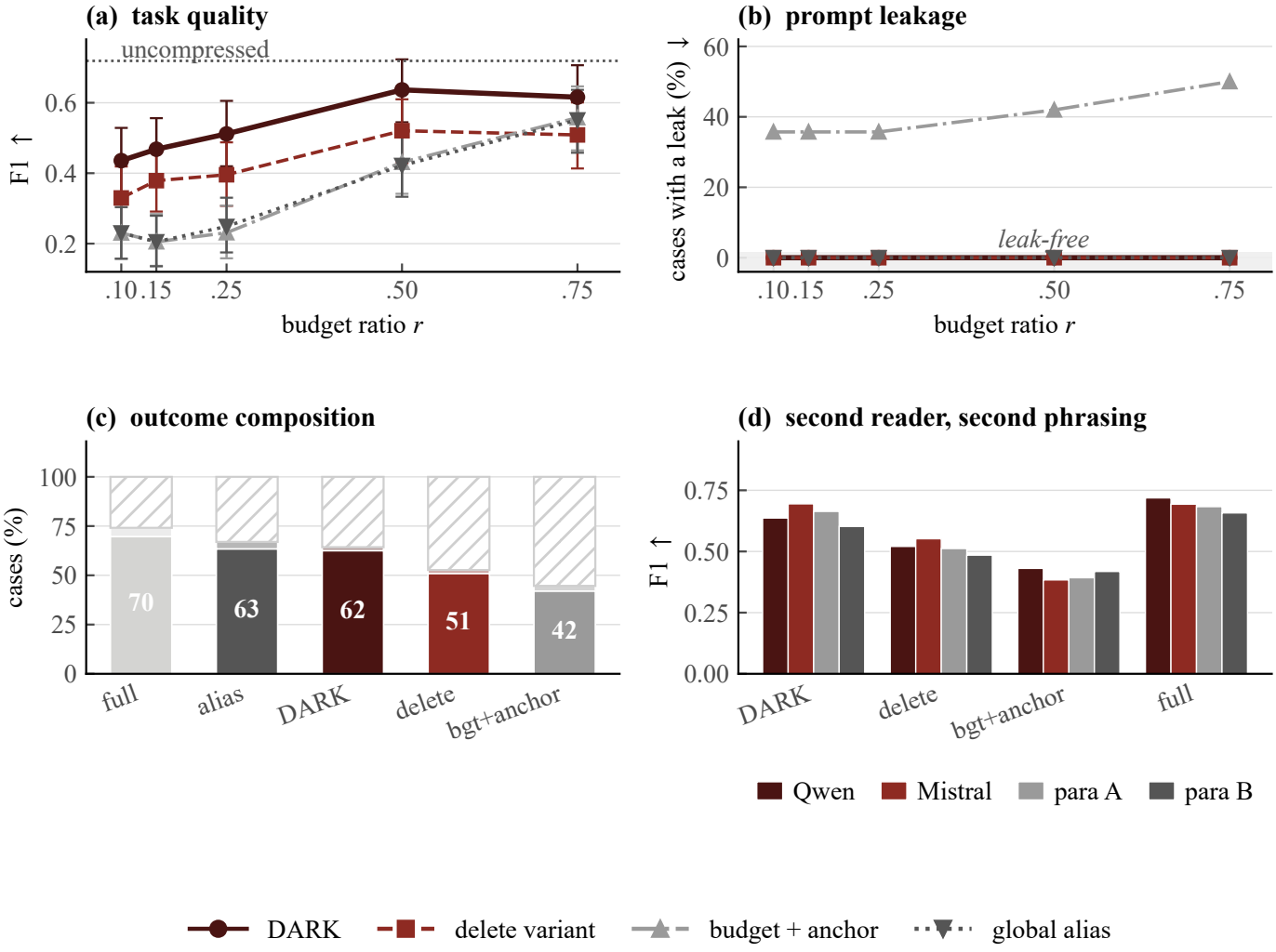


Fig. 5. Behaviour across the budget sweep (112 cases). (a) F1 at five ratios with bootstrap 95% intervals over cases; the dotted line is the uncompressed reference. (b) Fraction of cases in which a protected value is recoverable from the prompt. DARK, the delete variant and the global-alias arm stay at zero for the whole range, while the arm that compresses raw text leaks in 35.7% of cases at tight budgets and 50.0% at $r = 0.75$, so its apparent parity at loose budgets is bought with exposure. (c) Composition of case outcomes at $r = 0.50$: solid is an exact answer, the pale band is a partial match, hatched is a wrong or UNKNOWN answer. (d) The same four arms at $r = 0.50$ under the original reader, a second reader, and two mechanical rewritings of the question. DARK is the only arm that gains from the reader change (0.6366 → 0.6946) and draws level with the uncompressed arm there (0.6931); under both rewritings the ordering of the four arms is unchanged.

the answer, which is why the repair stops short of 100% rather than reaching it.

Figure 6 breaks the same question down by budget, by source, and by what the retention is worth. Panel (a) follows the answer record across the budget curve: the two arms that route on the record block hold it in 100% of the 112 cases from $r = 0.50$ upward and in 56.2% at $r = 0.10$, while the two arms that only protect the anchor line stay between 24.1% and 77.7% across the whole range. Panel (b) sets survival against F1 for the 2,728-case extension. Survival accounts for a good part of the ordering but not all of it: the delete variant keeps the answer record in 100% of cases and still scores 0.0000 on multi-entity cases, because the record it keeps no longer says which account it belongs to. The dissociation is not an artefact of the aggregate: panel (c) repeats the comparison within each of the six sources, and the block-level rule holds the answer in every case of every source while the redundancy-only rule falls to 16.7% on `toolmem_raw`. Panel (d) shows what that retention is worth per source; the one source where the block rule is behind, by −0.065, is the one where the redundancy-only arm retains 83.3% and the two are effectively tied.

B. The layout the rule depends on

The routing rule rests on two structural claims, and until now both were quoted as survival percentages rather than measured. We measured them from the case text alone: segment each excerpt on its record markers, take the first line that mentions the queried entity and the first line that carries the reference answer, and report their distance (Figure 7). No model is involved, so the measurement is independent of the reader.

TABLE V

ISOLATING THE ALIAS-INSTEAD-OF-DELETE RULE. REPLACING THE ALIAS OPERATION BY DELETION IN THE SENSITIVE-AND-RELEVANT CELL, WITH EVERYTHING ELSE HELD FIXED, COSTS A LARGE AND SIGNIFICANT AMOUNT WHEREVER THE ANCHOR IS ITSELF A SENSITIVE VALUE (MULTI-ENTITY, SINGLE-HOP, TWO-HOP) AND NOTHING WHERE IT IS NOT (TOPICAL VARIANTS, NO ANCHOR). THE TWO-HOP COLUMN IS THE CLEAREST CASE: THE REFERENCE ANSWER IS PRESENT IN THE CONTEXT IN 96.3% OF CASES UNDER THE DELETE VARIANT, AND F1 IS NEVERTHELESS 0.0000, BECAUSE THE ACCOUNT THAT IDENTIFIES THE RECORD HAS BEEN REMOVED.

Benchmark	DARK	delete variant	Δ	95% CI
extension, all (2,728)	0.4937	0.3142	$+0.1795$	$[+0.1643, +0.1945]$
multi-entity (1,500)	—	—	$+0.3273$	$[+0.3040, +0.3513]$
needle (1,228)	—	—	-0.0010	$[-0.0105, +0.0086]$ (n.s.)
main set (112)	0.6366	0.5205	$+0.1161$	$[+0.0446, +0.1875]$
LongBench topical (300)	0.3516	0.3503	$+0.0013$	$[-0.0002, +0.0042]$ (n.s.)
LongBench single-hop (300)	0.9607	0.1674	$+0.7933$	$[+0.7430, +0.8426]$
LongBench two-hop (300)	0.6733	0.0000	$+0.6733$	$[+0.6200, +0.7256]$

Both claims hold, and the split between the two case kinds is sharper than the aggregate suggested. On the 1,500 multi-entity cases the anchor and the answer always land in the same block—1,500 of 1,500—and never on the same line: the anchor sits on the private-contact lines and the answer inside the record line beneath it, two or five lines away in 87% of cases. On the 1,228 needle cases the answer is again inside the anchor's block in every case by annotation—an independent text-based re-derivation recovers 99.8%, the two exceptions being cases where the answer string also occurs in an earlier block—but in 56% of them it shares the anchor's line.

This asymmetry decides which of the two design choices carries the gain, and the two are not interchangeable. Widening the protected region from the anchor *line* to the whole *block* lifts F1 on both families—0.4802 to 0.6473 on needle, 0.2165 to 0.3487 on multi-entity—because in both the answer lives in the block and not always on the line. Switching the operation from deletion to aliasing, by contrast, pays only where the anchor is itself the protected value: 0.0000 to 0.3487 on multi-entity, and 0.6416 to 0.6473 on needle, where the record identifier that the question names is not a canary and survives either way.

Panel (c) is what separates the three design choices, because reading it left to right each step moves exactly one column and leaves the other two where they were. Replacing the raw text with aliases at a fixed selection takes raw exposure from 55.4% to 0% while answer retention stays at 57.6%. Widening the protected region from the line to the block takes answer retention from 57.6% to 100% while exposure stays at 0%. Deleting instead of aliasing inside that block leaves both columns untouched—the answer still survives and nothing is exposed—and instead takes collision from 0% to 100%, because every sensitive value becomes the same string and the reader can no longer tell which record the question is about. That single column is the whole difference between F1 0.4937 and 0.3142, and it is why the operation and not merely the region is load-bearing.

Panel (d) marks the boundary. Keeping the block is enough while the block is compact; past roughly twenty lines DARK's F1 on multi-entity cases falls from 0.370 to 0.257, and on needle cases from 0.724 to 0.395, because a large block spends the budget on itself and the answer drifts outside the fallback window. The rule does not fail gracefully here: it still emits a prompt within budget, and the loss appears only in the score.

C. Protecting the anchor line is not enough

A natural repair forbids dropping units that mention the queried entity. It does not close the gap, and where it helps it helps for the wrong reason. Overall retention moves from 47.5% to 57.6%, but the two families behave differently: multi-entity retention is unchanged (56.0% to 55.6%), while needle retention rises from 37.2% to 59.9%. Needle cases improve because the anchor there is the record identifier, which happens to share the record with the answer far more often than the account line does. The reason the repair stops short of 100% is layout: the line carrying the account identifier and the line carrying the answer are different units in 100% of multi-entity cases and in 65.4% of needle cases. Preserving the identifier preserves the pointer, not the content.

The repair becomes effective one level up. The excerpt is laid out in record blocks, and the block containing the anchor mention also contains the answer in every case of the benchmark, as its annotations record. Constraining the block instead of the line restores answer retention to 100% on both families without consulting the answer key. This is the measurement that motivates the routing rule: the relevant region is a block, not a unit.

D. The operation and the interface

Replacing the alias operation by deletion costs the task exactly where the design says it should. Holding the context construction fixed, the difference is large and significant where the anchor is itself a sensitive value: $+0.3273$ on multi-entity cases, $+0.7933$ on the single-hop variant, $+0.6733$ on the two-hop variant. Where no anchor exists—the topical variant, whose questions name no entity—it is $+0.0013$ and not significant. On the needle family, whose anchor is a non-sensitive record identifier, it is -0.0010 and not significant.

Figure 9 shows the two failure modes at the level of a single case, with the predictions the reader actually produced.

The two-hop column is the most informative single result in this paper. Under the delete variant the reference answer is

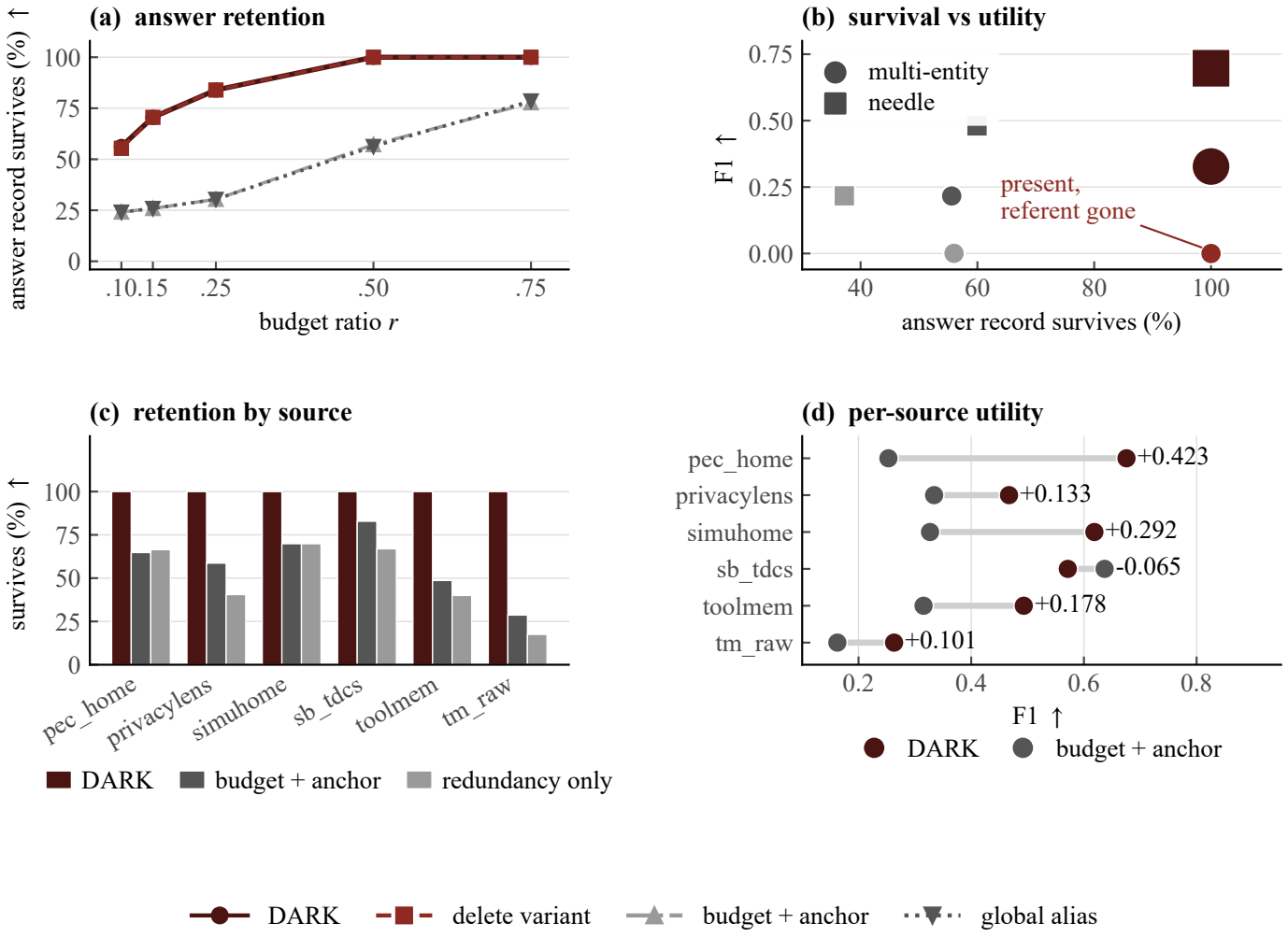


Fig. 6. Answer retention and what it does not explain. (a) Fraction of the 112 benchmark cases in which the record carrying the reference answer survives compression, as the budget varies. DARK and the delete variant coincide almost exactly: the alias-instead-of-delete rule does not change *which* units survive. (b) Survival against F1 on the 2,728-case extension; circles are multi-entity cases and squares are needle cases, and the four arms are the redundancy-only rule, the same rule with the anchor line protected, DARK, and the delete variant. The delete variant sits at 100% survival and 0.0000 F1 on multi-entity cases: the information is in the prompt and cannot be located. (c) Retention within each source at $r = 0.50$. (d) Per-source F1, DARK against the anchor-protected arm, with the paired difference printed at each row.

present in the context in 96.3% of cases, so GIC is essentially saturated, and F1 is 0.0000. The information is in the prompt; the reader cannot use it, because the identifier saying which record it belongs to has been removed. String-presence metrics do not detect this failure, which is why we report task quality and leakage as separate endpoints rather than as one utility score.

a) The question side.: Aliasing the context without rewriting the question produces a prompt whose halves use different names for the same entity. We measured both settings over the full 2,728 cases with the pipeline otherwise unchanged; Δ is the original-question score minus the rewritten one, so a negative value is what the rewrite is worth. On the subset where the question names a protected entity, dropping the rewrite takes the score to exactly zero.

Benchmark	rewritten	original	Δ	95% CI
extension (2,728)	0.4937	0.3132	−0.1806	[−0.195, −0.167]
multi-entity (1,500)	0.3273	0.0000	−0.3273	[−0.351, −0.303]
needle (1,228)	0.6970	0.6957	−0.0014	[−0.005, +0.002] (n.s.)

The damage is confined to the cases where the question names a protected entity. On multi-entity excerpts the unrewritten prompt scores exactly 0.0000: the question asks about `ACCT-4cd0` while every record offers `[ACCT-A]`, so no record can be matched and the reader cannot answer no matter what survives compression. On needle excerpts, where the question names a record identifier rather than an account, the step is worth 0.0014 and the interval includes zero. The other alias arms behave the same way: `alias_budget` loses 0.0444 overall and `alias_budget_link` 0.1272, the latter falling by 0.2314 on multi-entity and by exactly 0.0000 on needle.

The step costs nothing in privacy—leakage is 0% in both

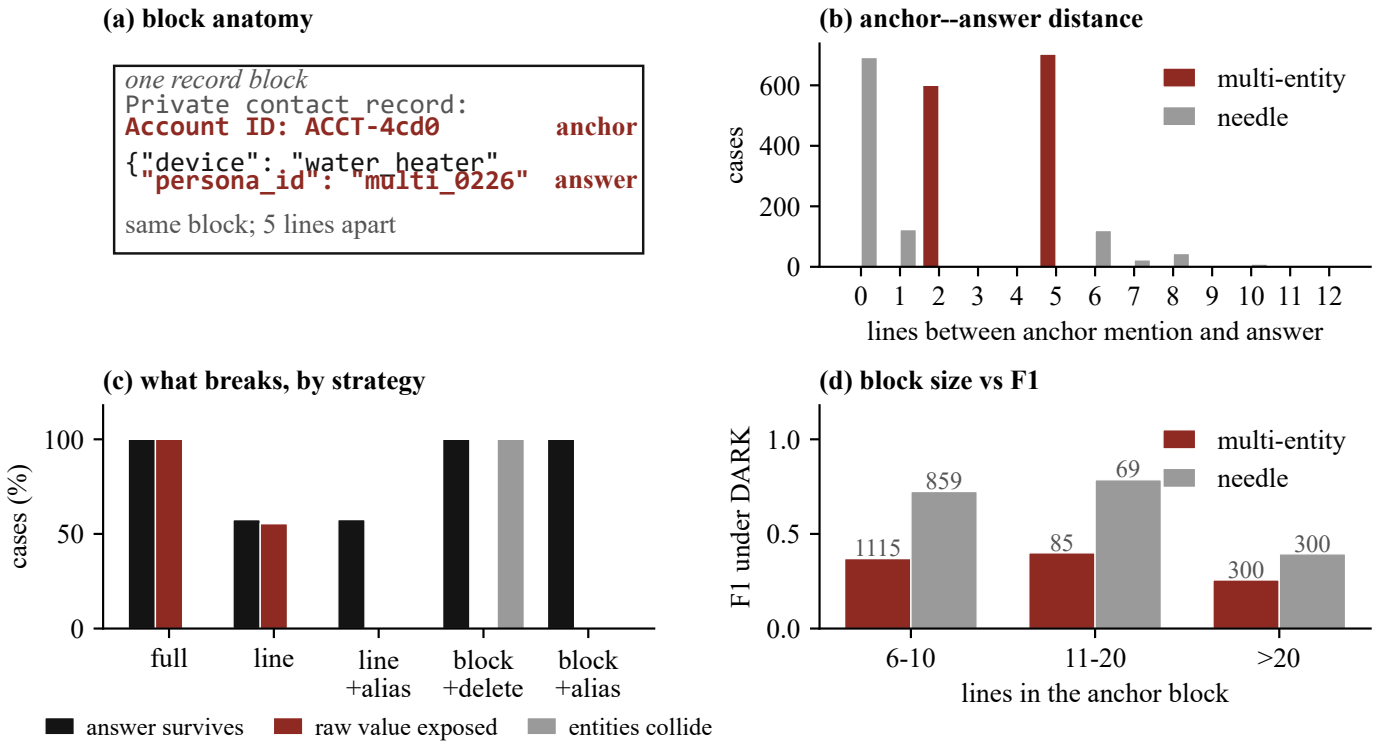


Fig. 7. The structure the rule depends on, measured from the case text without a model. (a) Inside one record block the anchor sits on the private-contact line and the answer inside the record line, five lines below it. (b) The anchor-answer line distance per case kind: the multi-entity cases are never closer than two lines, the needle cases usually share the line. (c) Five retention strategies scored on the only three things that can break—the answer span, exposure of the raw protected value, and collision of distinct entities onto one surface token. Reading left to right, each step of the design changes exactly one of the three columns. (d) DARK's F1 against the size of the block it has to keep; numbers above the bars are case counts. Once the anchor block exceeds twenty lines the rule degrades, which is where the fallback window stops being able to carry the answer.

conditions—so its entire value is recovered utility. We therefore treat the pair (q', P) rather than P alone as the object of study, and verify the mapping before generation.

E. What the answer key would add

A rule allowed to read the reference answer gains a great deal from it. On the full 2,728 cases, handing the answer span to four redundancy-based arms raises F1 by 0.157 to 0.305, and the gain is largest for the two anchor-protected arms—the arms used in the main comparison.

Arm	Δ F1 from answer key ↑	95% CI
BC	+0.1571	[+0.144, +0.171]
ALB	+0.1731	[+0.159, +0.188]
BCL	+0.3046	[+0.287, +0.322]
ABL	+0.3042	[+0.287, +0.323]

The two groups gain in different places, and the split is diagnostic. On needle cases the redundancy-only arms gain 0.348 and 0.349, because the answer value repeats across distractor records and knowing it tells the rule which copy to keep. On multi-entity cases those same arms gain almost nothing (0.001 and 0.029), while the anchor-protected arms gain 0.368 and 0.367—precisely the reverse. Each arm is helped where it was previously blind.

DARK reports every number in this paper without that input, because a deployed pipeline holds no reference answer. Stating the size here fixes the scale of what the rule obtains from the question alone: the referential benefit is reachable at inference time, not only in an evaluation that still holds the key. The oracle is not a privacy repair either: the anchor-protected arm forwards a protected value in 0% of cases with or without it, and the compress-only arm in 55.7% with it against 55.4% without.

F. Sensitivity and scope

Section IV-D left the traversal direction of the refill queue as an empirical choice. Running both directions end to end settles it at one boundary and leaves it open everywhere else.

Benchmark	increasing	decreasing	Δ	95% CI
extension (2,728)	0.4937	0.4832	−0.0106	[−0.028, +0.007]
main set (112)	0.6366	0.5844	−0.0522	[−0.124, +0.019]
topical (300)	0.3516	0.3172	−0.0344	[−0.090, +0.020]
single-hop (300)	0.9607	0.9719	+0.0111	[−0.008, +0.032]
two-hop (300)	0.6733	0.0167	−0.6567	[−0.712, −0.599]

For four of the five rows the interval includes zero: away from the boundary the order of refill units does not decide

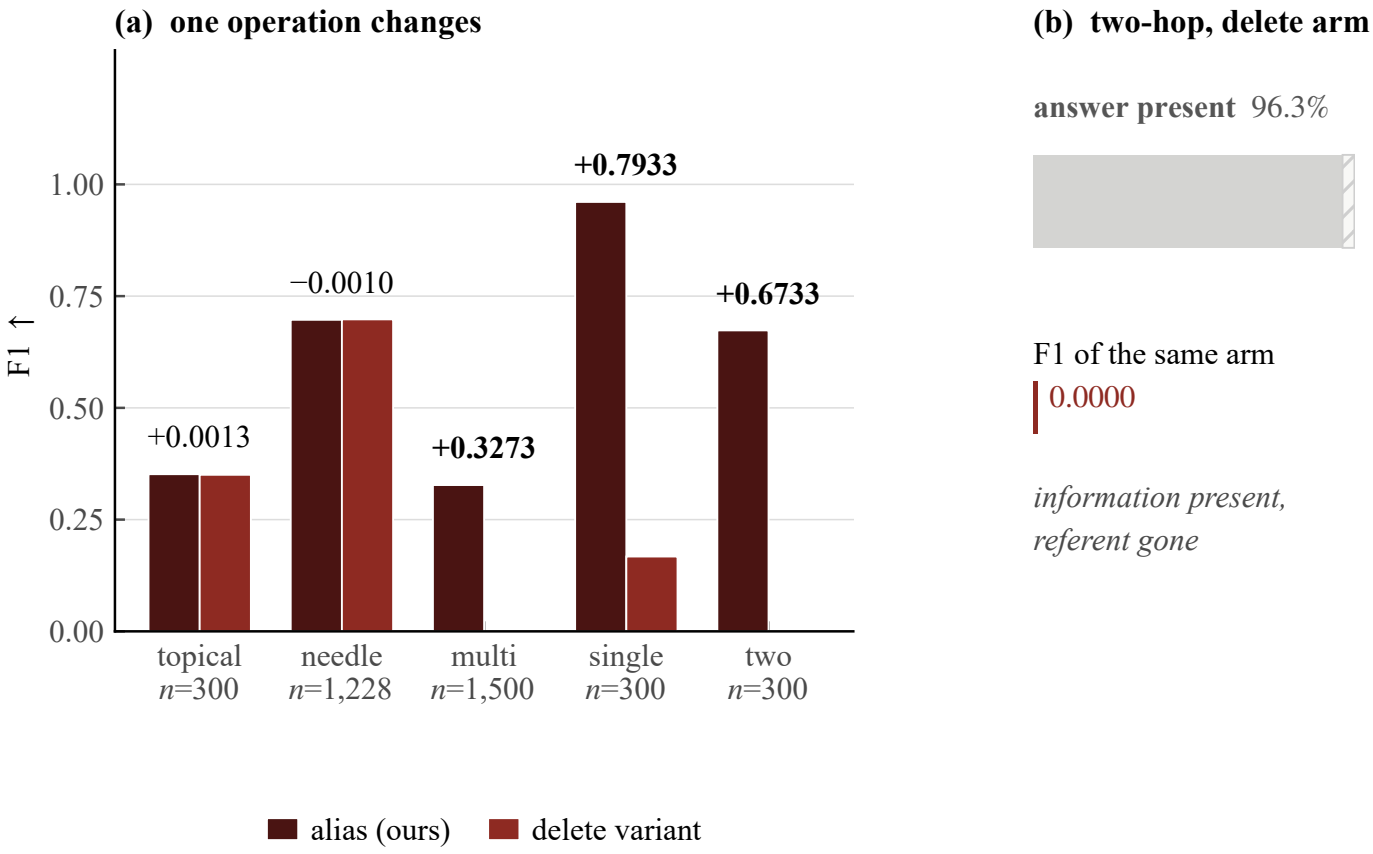


Fig. 8. Replacing the alias operation by deletion, with the context construction otherwise unchanged, on five settings. (a) The difference concentrates exactly where the anchor is a protected value and vanishes where it is not; labels give the paired difference of the two bars and a star marks an interval that excludes zero. (b) Distribution of the paired per-case difference for the three settings where the operation matters. The mass sits at both ends because a case is either answered or not, and the mean gain is largest on the single-hop variant. (c) In the two-hop variant the answer is still in the prompt after 96.3% of the deletions, and F1 is zero.

(a) two-hop: the answer sits behind a chain

Q: which device does account `ACCT-5f31` operate?

```
164  account ACCT-5f31 belongs to user U-5a675c
...
170  user U-5a675c operates device sensor_422708_15
```

after aliasing, both links keep a name:

```
account [ACCT-A] belongs to user [USER-B]
user [USER-B] operates device sensor_422708_15
```

(b) multi-entity: the answer looks like a distractor

Q: which persona id is recorded for account `ACCT-447f...`?

```
07  {"device": "dehumidifiers",
     "id": "multi_0166_00", ...}
08  Private contact record:
09  Contact email: contact.e67e...
11  Account ID: ACCT-447fa54ac66d
```

the target account is the eleventh of sixteen,
and its record is shaped like every other record.

	full	alias	delete	DARK
(a) two-hop, `lbc-hotpotqa-0000`	sensor_422708_15	sensor_422708_15	UNKNOWN	sensor_422708_15
(b) multi-entity, `multi-0032`	UNKNOWN	multi_0166_00	UNKNOWN	multi_0166_00

Fig. 9. Two cases where the routing changes the answer, with the predictions the reader actually produced. In (a) the question names an account that reaches the answer only through a user identifier six records later; deleting either link leaves the answer present but unreachable, and the model returns UNKNOWN. In (b) the answer sits in one of sixteen near-identical account records, so the uncompressed prompt *also* fails—here the distractor structure, not the budget, is what defeats the reader—while both aliasing arms answer correctly. Shaded spans mark the names that keep a chain addressable.

whether the answer survives. The two-hop benchmark *is* the boundary. There the answer sits six sentences from the anchor, outside the fallback window, so whether it is present at all is decided by which units refill admits, and keeping the repetitive ones retrieves it (0.6733 against 0.0167). We read this as a property of this data—the anti-correlation between redundancy and answer-bearing measured at the start of this section—rather than as a guarantee, which is why Section IX proposes widening

the relevant region instead of relying on the ordering.

a) Rule-based compressors on records.: A 2026 line compresses without a model call at all: deterministic lexical rules delete filler, contractions, stop words and function words, then lemmatise what remains [21]. We ran its most aggressive preset over all 2,728 cases on CPU. It realises 5,817 context tokens against our 4,146 and the uncompressed 8,482, so it compresses less than we do, and it scores 0.3696 F1 against our 0.4937, a paired difference of -0.1242 $[-0.1462, -0.1016]$ at 100% leakage against our 0%. The mechanism is visible in the text. The rules strip braces and rename keys—`device` to `twist` in one case—so the number of parseable JSON objects in the corpus falls from 273,209 of 299,408 (91.2%) to zero, and the reader receives a word salad in which the record boundaries that anchor the question no longer exist. A compressor tuned on prose is not a compressor for records.

b) Where the rule applies.: The three LongBench-derived variants locate the boundary.

Variant	`full`	`alias`	DARK
topical (300)	0.4181	0.4074	0.3516
single-hop (300)	0.9889	0.9197	0.9607
two-hop (300)	0.0967	0.7078	0.6733

On the topical variant the questions name no entity, so there is nothing to route on: the rule degenerates to global aliasing plus redundancy ordering and loses 0.0665 against the uncompressed prompt. That variant lies outside what this rule addresses, and we report it as such rather than as a failure of the mechanism.

The two-hop row carries a second observation. Without sanitization the raw prompt scores 0.0967; with global aliasing it scores 0.7078. The identifiers in this variant are long hexadecimal strings differing in a few characters, so replacing them with short distinct tokens makes the task *easier* for the reader. Privacy and utility move in the same direction here, a useful counterpoint to the usual framing, though the effect belongs to the identifier surface form rather than to the routing rule.

IX. Discussion

a) What the study establishes.: For excerpts of IoT-style operational records, queried by a question that names an entity whose value must not reach the model, the record that answers the question and the record that stores the value are the same block. Routing each unit on (sensitivity, relevance) and aliasing the sensitive-and-relevant cell therefore preserves the task at half the tokens and at zero measured leakage, without a reference answer and without touching the reader. The claim is that specific: it is a statement about queries whose referent is also the protected value, in a layout where the two coincide.

b) Where the mechanism stops.: The three variants in Section VIII delimit it. The rule is unnecessary when the question is topical and names no entity, since there is then no referent to protect or to recover. It is redundant when anchor and answer coincide in a single record, where protecting the anchor line already suffices. It depends on the refill order when the answer lies outside the fallback window, the one place where the ordering rather than the routing decides the outcome. We do not claim that it improves ordinary compression on ordinary prompts, and the topical variant shows that it does not.

TABLE VI

Where the rule stops and what is not measured. Each row is a condition under which the rule is expected to weaken or to be the wrong tool; *measured* gives the endpoint we have for it in this paper, and a dash means we did not run that configuration. We list the unmeasured rows on purpose: they bound what the evidence supports more than the measured ones do.

Condition	Current behaviour	Measured
Question names no entity (topical)	rule not needed	$+0.0013$ (n.s.)
Anchor is not itself protected	operation barely matters	0.6416 vs 0.6473
Answer outside the fallback window	refill order decides	0.6733 two-hop
Anchor block larger than 20 lines	block spends the budget	0.257 multi
Anchor and answer in different blocks	routing keeps wrong block	—
Two entities share one identifier	anchor is ambiguous	—
Alias map disclosed to the reader	full recovery by design	—
Many prompts seen by one provider	cross-prompt linkage	—

c) The adversary we assume.: The adversary is the model provider: it receives the prompt, may retain it, and may hold many prompts from the same deployment. It does not hold the alias map, does not observe the raw log, and does not observe the local construction, which runs inside the operator's boundary. Under that model the quantity we bound is the recoverability of a protected value from the prompt text, and the routing rule reduces it to zero on this benchmark.

Three properties of the construction belong in this paragraph rather than in the evidence, because they are choices and not measurements. Aliases are assigned per prompt, not from a global table: the value pool in Section V is small enough that a handle recurs in roughly twenty cases, so a global pseudonym would let one provider link those cases to each other. Per-prompt assignment removes that link at the cost of making the map prompt-local. The assignment is keyed, which removes the rank attack that an unkeyed assignment admits, but a keyed assignment is still an assignment: whoever holds the key recovers every value. And nothing here prevents the reader from *inferring* a protected attribute that the context still implies, because answering such questions is the task.

d) What the privacy claim covers.: Leakage is measured as recoverability of the protected values from the exact prompt text, using the pattern set of the published protocol (full value plus fragments). That is the relevant quantity for a pipeline whose adversary is the model provider and whose prompts may be retained. It is not a formal guarantee. We derive no differential-privacy bound [36] and no k-anonymity guarantee [35]; the alias map is assumed private, and an adversary who obtains it recovers the values. Keying the assignment from a permutation removes the rank attack that an unkeyed assignment admits—an adversary holding only the value set can otherwise invert the map—but that mitigation is narrower than a guarantee. We do not evaluate inference attacks

either: membership, attribute inference, and reconstruction of a value from its alias's co-occurrence pattern are all outside the present evidence.

The measured benefit of keeping repetitive units follows from an anti-correlation between redundancy and answer-bearing in this data. It is not derived from the definition of the score, and a corpus whose answers are unique rather than repeated would not exhibit it. The principled fix is to make the relevant region cover the answer instead of relying on the ordering. In the two-hop variant the answer lies at a fixed distance of six sentences from the anchor, so a window of that width would protect it explicitly and remove the dependence. We have not run that configuration, and the present results should be read as depending on the fallback width.

Table VI collects the conditions under which the rule weakens. Four of the eight rows are measured; we list the other four because they bound the claim more than the measured ones do. The two that a reader should weigh first are the cross-prompt one, which our threat model assumes away rather than defeats, and the block-size one, where the rule does not fail gracefully: it still returns a prompt inside budget and the loss appears only in the score.

e) Cost and threats.: Prompt construction is lexical and model-free: one pass over the units to label them, one pass to score redundancy (quadratic in the number of units in the worst case), and one pass to fit the budget. It adds no model call. On the longest excerpt in the extension set (19,838 tokens uncompressed) construction completes in under a second on CPU. We did not measure end-to-end wall-clock or memory of the reader, so the efficiency we claim concerns prompt length rather than latency.

The protected values are synthetic canaries injected into public corpora, because those corpora cannot carry real personal data. Canary leakage is measurable exactly, but the measured rate is a property of this construction. Both benchmark sizes are generated by our own scripts from shared templates, so the questions are entity-resolving by construction and the two-hop distance is fixed rather than sampled. The readers are 7B-parameter instruction-tuned models, and we have no evidence about larger or API-served models. Finally, the two benchmark sizes reuse case identifiers for different content, so a cross-set join by identifier would silently mix records; we guard against this by verifying the uncompressed arm against a per-set reference value, but the hazard is worth stating for anyone reusing the artifacts.

X. Conclusion

We studied what happens to a privacy-preserving inference pipeline when the entity a question asks about is also the entity that must be protected. Both standard remedies fail there, for opposite reasons: redaction removes the referent the reader needs, and redundancy ordering removes the record that carries the answer at close to chance, because the score ranks units by self-similarity and knows nothing about which record the question is about. DARK resolves the conflict by deciding the two axes per unit and giving the sensitive-and-relevant cell the one operation neither remedy provides: the unit stays, its values are aliased, and the same map is written into the question. Relevance is read from the question—the record block that mentions the queried entity—so the pipeline needs neither a reference answer nor training.

Across a 112-case benchmark over six public datasets, DARK keeps 48.6% of the tokens at zero prompt-level leakage and is statistically indistinguishable from aliasing the full context without compression. On a 2,728-case extension it matches the strongest query-aware compressor at 0% leakage against 98.1%, and exceeds the nearest leak-free baseline by 0.3226 F1 points. The 2026 rule-based compressor we re-run needs no model at all, yet costs more tokens than we spend and still leaks in every case, because its rules delete the record syntax the question depends on. Replacing the alias operation with deletion is what moves those numbers: it costs 0.1795 F1 overall, 0.3273 on multi-entity cases, and nothing where no anchor exists, so the rule acts on the structure it was designed for.

Two questions remain open. The fallback window is the weakest part of the mechanism: when the answer lies outside it, as in the two-hop variant, recovery depends on the refill order, and widening the relevant region to cover the answer is the first configuration to test. The second concerns the privacy claim. A pipeline of this shape would be considerably stronger with a guarantee that survives an adversary holding the alias map, and establishing one is a different piece of work from the one reported here.

References

[1] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[2] H. Jiang, Q. Wu, X. Luo, D. Zhang, C.-Y. Lin, and Y. Yang, "LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[3] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, "LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[4] Y. Li, B. Dong, F. Guerin, and C. Lin, "Compressing context to enhance inference efficiency of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[5] N. Chuang *et al.*, "Provence: Efficient and robust context pruning for retrieval-augmented generation," in *International Conference on Learning Representations (ICLR)*, 2025.

[6] B. Liskavets, M. Ushakov, S. Roy, M. Klibanov, A. Etemad, and S. Luke, "Prompt compression with context-aware sentence encoding for fast and improved LLM inference," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, arXiv:2409.01227.

[7] Y. Wang, X. Huang, B. Tian, Y. Su, L. Yu, H. Liao, Y. Fan, J. Guo, and X. Cheng, "QUITO-X: A new perspective on context compression from the information bottleneck theory," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 6841–6856.

[8] Y. Zhao, Z. Li, H. Zhao, B. Qi, and G. Liu, "DAC: A dynamic attention-aware approach for task-agnostic prompt compression," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 19 395–19 407.

[9] J. Tang, J. Xu, T. Lu, Z. Zhang, Y. Zhao, L. Hai, and H.-T. Zheng, "Perception compressor: A training-free prompt compression framework in long context scenarios," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 4093–4108.

[10] J. Mu, X. Li, and N. Goodman, "Learning to compress prompts with gist tokens," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[11] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, and F. Wei, "In-context autoencoder for context compression in a large language model," in *International Conference on Learning Representations (ICLR)*, 2024.

[12] A. Chevalier, A. Wettig, A. Ajith, and D. Chen, "Adapting language models to compress contexts," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 3829–3846.

[13] X. Cheng, X. Wang, X. Zhang, T. Ge, S.-Q. Chen, F. Wei, H. Zhang, and D. Zhao, "xRAG: Extreme context compression for retrieval-augmented generation with one token," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

[14] S. Ren, Q. Jia, and K. Zhu, "Context compression for auto-regressive transformers with sentinel tokens," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 12 860–12 867.

[15] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation," in *International Conference on Learning Representations (ICLR)*, 2024.

[16] J. Liu, L. Li, T. Xiang, B. Wang, and Y. Qian, "TCRA-LLM: Token compression retrieval augmented large language model for inference cost reduction," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 9796–9810.

[17] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H_2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[18] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "SnapKV: LLM knows what you are looking for before generation," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

[19] W. Łajewska, M. Hardalov, L. Aina, N. A. John, H. Su, and L. Marquez, "Understanding and improving information preservation in prompt compression for LLMs," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 17 520–17 541.

[20] J. Ma, Z. Feng, E. Chersoni, and S. Chen, "Every time I hire a linguist, inference costs go down: On linguistic rules as effective prompt compressors," in *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Budapest, Hungary, 2026, pp. 11 714–11 750.

[21] S. Chokshi, "Lexical prompt compression for large language models: A training-free, deterministic pipeline with empirical Pareto analysis across eleven task categories," arXiv:2609.13154 [cs.CL], 2026, preprint, 8 July 2026.

[22] J. Zhang, Z. Tian, M. Zhu, Y. Song, T. Sheng, S. Yang, Q. Du, X. Liu, M. Huang, and D. Li, "DYNTEXT: Semantic-aware dynamic text sanitization for privacy-preserving LLM inference," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 20 243–20 255.

[23] I. Pilán, B. Manzanares-Salor, D. Sánchez, and P. Lison, "Truthful text sanitization guided by inference attacks," *Applied Soft Computing*, p. 114013, 2025.

[24] Y. Xiao, Y. Jin, Y. Bai, Y. Wu, X. Yang, X. Luo, W. Yu, X. Zhao, Y. Liu, Q. Gu, H. Chen, W. Wang, and W. Cheng, "Large language models can be contextual privacy protection learners," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 14 179–14 201.

[25] L. Siyan, V. C. Raghuram, O. Khattab, J. Hirschberg, and Z. Yu, "PAPILLON: Privacy preservation from Internet-based and local language model ensembles," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL), Volume 1: Long Papers*, 2025, pp. 3371–3390.

[26] J. Zhan, W. Zhang, Z. Zhang, H. Xue, Y. Zhang, and Y. Wu, "Portcullis: A scalable and verifiable privacy gateway for third-party LLM inference," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025, pp. 1022–1030.

[27] Y. Cheng, L. Zhang, J. Wang, M. Yuan, and Y. Yao, "RemoteRAG: A privacy-preserving LLM cloud RAG service," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 3820–3837.

[28] S. Kim, S. Yun, H. Lee, M. Gubri, S. Yoon, and S. J. Oh, "ProPILE: Probing privacy leakage in large language models," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[29] R. Staab, M. Vero, M. Balunovic, and M. Vechev, "Beyond memorization: Violating privacy via inference with large language models," in *International Conference on Learning Representations (ICLR)*, 2024.

[30] R. Xie, J. Wang, R. Huang, M. Zhang, R. Ge, J. Pei, N. Z. Gong, and B. Dhingra, "ReCaLL: Membership inference via relative conditional log-likelihoods," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 8671–8689.

[31] Y. Zhou, Z. Zhang, and X. Zhang, "Exploiting the shadows: Unveiling privacy leaks through lower-ranked tokens in large language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2025, pp. 8376–8386.

[32] E. Debenedetti, G. Severi, N. Carlini, C. A. Choquette-Choo, M. Jagielski, M. Nasr, E. Wallace, and F. Tramèr, "Privacy side channels in machine learning systems," in *33rd USENIX Security Symposium (USENIX Security)*, 2024.

[33] P. Mai, R. Yan, Z. Huang, Y. Yang, and Y. Pang, "Split-and-denoise: Protect large language model inference with local differential privacy," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[34] B. C. Das, M. H. Amini, and Y. Wu, "Security and privacy challenges of large language models: A survey," *ACM Computing Surveys*, vol. 57, no. 6, pp. 1–39, 2025.

[35] T. Carvalho, N. Moniz, P. Faria, and L. Antunes, "Survey on privacy-preserving techniques for microdata publication," *ACM Computing Surveys*, vol. 55, no. 14s, pp. 1–42, 2023.

[36] Y. Zhao, J. T. Du, and J. Chen, "Scenario-based adaptations of differential privacy: A technical survey," *ACM Computing Surveys*, vol. 56, no. 8, pp. 1–39, 2024.

[37] K. Edemacu and X. Wu, "Privacy preserving prompt engineering: A survey," *ACM Computing Surveys*, vol. 57, no. 10, pp. 1–36, 2025.

[38] Z. Shen, Z. Xi, Y. He, W. Tong, J. Hua, and S. Zhong, "ProSan: Utility-based prompt privacy sanitizer," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 1198–1212, 2026.
[39] Z. Zaman, W. Xue, P. Gauravaram, W. Hu, J. Jiang, and S. K. Jha, "Privacy-preserving probabilistic data encoding for IoT data analysis," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 9173–9187, 2024.
[40] Y. Gao, L. Chen, J. Han, G. Wu, and W. Susilo, "IoT privacy-preserving data mining with dynamic incentive mechanism," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 777–790, 2024.
[41] X. Yan, Y. Miao, X. Li, K.-K. R. Choo, X. Meng, and R. H. Deng, "Privacy-preserving asynchronous federated learning framework in distributed IoT," *IEEE Internet of Things Journal*, vol. 10, no. 15, pp. 13 281–13 291, 2023.
[42] Y. Zhang, L. Chen, Y. Zhu, and X. Kong, "Privacy-enhanced role-based access control for IoT systems," *IEEE Internet of Things Journal*, vol. 12, no. 14, pp. 27 269–27 279, 2025.
[43] J. Sun, Y. Bao, W. Qiu, R. Lu, S. Zhang, Y. Guan, and X. Cheng, "Privacy-preserving fine-grained data sharing with dynamic service for the cloud-edge IoT," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 2, pp. 1329–1346, 2025.
[44] Z. Wang, Y. Sun, D. Liu, J. Hu, X. Pang, Y. Hu, and K. Ren, "Location privacy-aware task offloading in mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 23, no. 3, pp. 2269–2283, 2024.
[45] J. Tian, M. Li, Z. Wang, L. Chen, J. Qin, and R. Zhang, "OMLog: Online log anomaly detection for evolving system with meta-learning," *IEEE Internet of Things Journal*, vol. 12, no. 15, pp. 30 142–30 155, 2025.
[46] J. Zhou, S. Ying, S. Wang, D. Zhao, J. Xiang, K. Liang, and P. Liu, "LogDLR: Unsupervised cross-system log anomaly detection through domain-invariant latent representation," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 4, pp. 4456–4471, 2025.
[47] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
[48] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li, "LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2025, pp. 3639–3664.
[49] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, and B. Ginsburg, "RULER: What's the real context size of your long-context language models?" in *Conference on Language Modeling (COLM)*, 2024.
[50] H. Yen, T. Gao, M. Hou, K. Ding, D. Fleischer, P. Izsak, M. Wasserblat, and D. Chen, "HELMET: How to evaluate long-context models effectively and thoroughly," in *International Conference on Learning Representations (ICLR)*, 2025.
[51] X. Zhang, Y. Chen, S. Hu, Z. Xu, J. Chen, M. Hao, X. Han, Z. Thai, S. Wang, Z. Liu, and M. Sun, "∞Bench: Extending long context evaluation beyond 100K tokens," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2024, pp. 15 262–15 277.
[52] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
[53] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

APPENDIX

A. Per-source breakdown

*a) Extension set (*2,728 *cases).:* F1 by source, budget $r = 0.50$, all arms of ours constructed without the answer key. The routing rule is strongest on `pec_home` and `smartbench_tdcs` and weakest on `toolmem_raw`, where the uncompressed arm is also weakest.

TABLE VII
Per-source F1 on the extension set (2,728 cases, $r = 0.50$).

Source	n	`full`	DARK	ABL
`pec_home`	426	0.7816	0.6756	0.2859
`privacylens`	600	0.5340	0.4671	0.3381
`simuhome`	312	0.8686	0.6186	0.3910
`smartbench_tdcs`	490	0.6449	0.5714	0.6367
`toolmem`	300	0.5154	0.4933	0.3121
`toolmem_raw`	600	0.4401	0.2631	0.1624
overall	2728	0.6081	0.4937	0.3482

*b) Main set (*112 *cases).:* Same setting on the smaller set.

TABLE VIII
Per-source F1 on the main set (112 cases, $r = 0.50$).

Source	n	`full`	DARK	`alias`
`pec_home`	20	0.8750	0.8250	0.8250
`privacylens`	20	0.6567	0.6400	0.4900
`simuhome`	20	0.8500	0.7500	0.7500
`smartbench_tdcs`	20	0.7000	0.7000	0.6500
`toolmem`	12	0.4901	0.4167	0.4354
`toolmem_raw`	20	0.6500	0.4000	0.6500
overall	112	0.7189	0.6366	0.6475

B. Anchor types covered

The anchor key is resolved differently depending on what the question names. All six forms occurring in the extension set are handled, and the anchor string is present in the context in every case (2,728/2,728).

TABLE IX
Anchor key forms covered by the routing rule. All six occur in the extension set and the anchor string is present in the context in every case (2,728/2,728).

`anchor_kind`	Example question form	n
none (entity field)	*for account* `ACCT-...`	1500
`id`	*for record id* `pec_home_...`	316
`name`	names an entity directly	300
`session`	names a session	300
`entry`	*for entry 3*	281
`native`	native identifier	31

C. Leakage counting

A case counts as leaking when any protected value, or a fragment of it, is recoverable from the concatenation of the rewritten question and the compressed context. Fragments are the pattern set of the published protocol. The count is per case, not per value, so the reported rate is the fraction of cases with at least one exposure; the per-value count is retained in the artifacts.

D. Reproduction notes

Every table in this paper is computed from per-case records that ship with the artifacts, one line per (case, arm). The uncompressed arm in each set is used as an identity check: a result set whose uncompressed arm does not reproduce the reference value is not part of this paper. Budget arms assert non-overshoot per case; a run that violates it is discarded rather than truncated. All arms of ours are keyed alias runs; an unkeyed assignment changes both the alias strings and the resulting score, so results are only comparable within one setting.